

 Introduction:

Loantap is a leading financial technology company based in India, specializing in providing flexible and innovative loan products to individuals and businesses. With a focus on customer-centric solutions, Loantap leverages technology to offer hassle-free borrowing experiences, including personal loans, salary advances, and flexible EMI options. Their commitment to transparency, speed, and convenience has established them as a trusted partner for borrowers seeking efficient financial solutions.

- LoanTap is at the forefront of offering tailored financial solutions to millennials.
- Their innovative approach seeks to harness data science for refining their credit underwriting process.
- The focus here is the Personal Loan segment. A deep dive into the dataset can reveal patterns in borrower behavior and creditworthiness.
- Analyzing this dataset can provide crucial insights into the financial behaviors, spending habits, and potential risk associated with each borrower.
- The insights gained can optimize loan disbursal, balancing customer outreach with risk management.

 Our Task:

- As a data scientist at LoanTap, you are tasked with analyzing the dataset to determine the creditworthiness of potential borrowers. Your ultimate objective is to build a logistic regression model, evaluate its performance, and provide actionable insights for the underwriting process.

 Features of the dataset:

- Column Profiling:

Feature	Description
loan_amnt	The listed amount of the loan applied for by the borrower. If at some point in time, the credit department reduces the loan amount, then it will be reflected in this value
term	The number of payments on the loan. Values are in months and can be either 36 or 60
int_rate	Interest Rate on the loan
installment	The monthly payment owed by the borrower if the loan originates
grade	LoanTap assigned loan grade
sub_grade	LoanTap assigned loan subgrade
emp_title	The job title supplied by the Borrower when applying for the loan
emp_length	Employment length in years. Possible values are between 0 and 10 where 0 means less than one year and 10 means ten or more years
home_ownership	The home ownership status provided by the borrower during registration or obtained from the credit report
annual_inc	The self-reported annual income provided by the borrower during registration
verification_status	Indicates if income was verified by LoanTap, not verified, or if the income source was verified
issue_d	The month which the loan was funded
loan_status	Current status of the loan - Target Variable
purpose	A category provided by the borrower for the loan request
title	The loan title provided by the borrower
dti	A ratio calculated using the borrower's total monthly debt payments on the total debt obligations, excluding mortgage and the requested LoanTap loan, divided by the borro
earliest_cr_line	The month the borrower's earliest reported credit line was opened
open_acc	The number of open credit lines in the borrower's credit file
pub_rec	Number of derogatory public records
revol_bal	Total credit revolving balance
revol_util	Revolving line utilization rate, or the amount of credit the borrower is using relative to all available revolving credit
total_acc	The total number of credit lines currently in the borrower's credit file
initial_list_status	The initial listing status of the loan
application_type	Indicates whether the loan is an individual application or a joint application with two co-borrowers
mort_acc	Number of mortgage accounts
pub_rec_bankruptcies	Number of public record bankruptcies
Address	Address of the individual

▼ Exploratory Data Analysis

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import ttest_ind,chi2_contingency

from sklearn.linear_model import LogisticRegression
```

```
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import MinMaxScaler, LabelEncoder, StandardScaler
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    roc_auc_score, roc_curve, auc, precision_recall_curve, average_precision_score,
    ConfusionMatrixDisplay, RocCurveDisplay, f1_score, recall_score, precision_score
)

from statsmodels.stats.outliers_influence import variance_inflation_factor
from imblearn.over_sampling import SMOTE

import warnings
warnings.filterwarnings("ignore")

!gdown 1ZPYj7CZCfxntE8p2Lze_4Q04MyE0y6_d
df= pd.read_csv('logistic_regression.csv')
df.head()
```

Downloading...
From: https://drive.google.com/uc?id=1ZPYj7CZCfxntE8p2Lze_4Q04MyE0y6_d
To: /content/logistic_regression.csv
100% 100M/100M [00:00<00:00, 124MB/s]

	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ownership	annual_inc	verificat
0	10000.0	36 months	11.44	329.48	B	B4	Marketing	10+ years	RENT	117000.0	
1	8000.0	36 months	11.99	265.68	B	B5	Credit analyst	4 years	MORTGAGE	65000.0	
2	15600.0	36 months	10.49	506.97	B	B3	Statistician	< 1 year	RENT	43057.0	S
3	7200.0	36 months	6.49	220.65	A	A2	Client Advocate	6 years	RENT	54000.0	
4	24375.0	60 months	17.27	609.33	C	C5	Destiny Management Inc.	9 years	MORTGAGE	55000.0	

```
pd.set_option('display.max_columns', None)
```

📉🔍 Exploration of data :

```
df.shape
```

➡️ (396030, 27)

```
df.info()
```

➡️ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 396030 entries, 0 to 396029
Data columns (total 27 columns):
Column Non-Null Count Dtype

0 loan_amnt 396030 non-null float64
1 term 396030 non-null object
2 int_rate 396030 non-null float64
3 installment 396030 non-null float64
4 grade 396030 non-null object
5 sub_grade 396030 non-null object
6 emp_title 373103 non-null object
7 emp_length 377729 non-null object
8 home_ownership 396030 non-null object
9 annual_inc 396030 non-null float64
10 verification_status 396030 non-null object
11 issue_d 396030 non-null object
12 loan_status 396030 non-null object
13 purpose 396030 non-null object
14 title 394274 non-null object
15 dti 396030 non-null float64
16 earliest_cr_line 396030 non-null object
17 open_acc 396030 non-null float64
18 pub_rec 396030 non-null float64
19 revol_bal 396030 non-null float64
20 revol_util 395754 non-null float64
21 total_acc 396030 non-null float64
22 initial_list_status 396030 non-null object

```
23 application_type      396030 non-null object
24 mort_acc              358235 non-null float64
25 pub_rec_bankruptcies  395495 non-null float64
26 address               396030 non-null object
dtypes: float64(12), object(15)
memory usage: 81.6+ MB
```

df.columns

```
Index(['loan_amnt', 'term', 'int_rate', 'installment', 'grade', 'sub_grade',
      'emp_title', 'emp_length', 'home_ownership', 'annual_inc',
      'verification_status', 'issue_d', 'loan_status', 'purpose', 'title',
      'dti', 'earliest_cr_line', 'open_acc', 'pub_rec', 'revol_bal',
      'revol_util', 'total_acc', 'initial_list_status', 'application_type',
      'mort_acc', 'pub_rec_bankruptcies', 'address'],
      dtype='object')
```

Statistical Summary

df.describe().T

	count	mean	std	min	25%	50%	75%	max
loan_amnt	396030.0	14113.888089	8357.441341	500.00	8000.00	12000.00	20000.00	40000.00
int_rate	396030.0	13.639400	4.472157	5.32	10.49	13.33	16.49	30.99
installment	396030.0	431.849698	250.727790	16.08	250.33	375.43	567.30	1533.81
annual_inc	396030.0	74203.175798	61637.621158	0.00	45000.00	64000.00	90000.00	8706582.00
dti	396030.0	17.379514	18.019092	0.00	11.28	16.91	22.98	9999.00
open_acc	396030.0	11.311153	5.137649	0.00	8.00	10.00	14.00	90.00
pub_rec	396030.0	0.178191	0.530671	0.00	0.00	0.00	0.00	86.00
revol_bal	396030.0	15844.539853	20591.836109	0.00	6025.00	11181.00	19620.00	1743266.00
revol_util	395754.0	53.791749	24.452193	0.00	35.80	54.80	72.90	892.30
total_acc	396030.0	25.414744	11.886991	2.00	17.00	24.00	32.00	151.00
mort_acc	358235.0	1.813991	2.147930	0.00	0.00	1.00	3.00	34.00
pub_rec_bankruptcies	395495.0	0.121648	0.356174	0.00	0.00	0.00	0.00	8.00

df.describe(include='object').T

	count	unique	top	freq
term	396030	2	36 months	302005
grade	396030	7	B	116018
sub_grade	396030	35	B3	26655
emp_title	373103	173105	Teacher	4389
emp_length	377729	11	10+ years	126041
home_ownership	396030	6	MORTGAGE	198348
verification_status	396030	3	Verified	139563
issue_d	396030	115	Oct-2014	14846
loan_status	396030	2	Fully Paid	318357
purpose	396030	14	debt_consolidation	234507
title	394274	48816	Debt consolidation	152472
earliest_cr_line	396030	684	Oct-2000	3017
initial_list_status	396030	2	f	238066
application_type	396030	3	INDIVIDUAL	395319
address	396030	393700	USCGC Smith\nFPO AE 70466	8

Duplicate Detection

df[df.duplicated()]

loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ownership	annual_inc	verification
-----------	------	----------	-------------	-------	-----------	-----------	------------	----------------	------------	--------------

▼ Insights

- The dataset does not contain any duplicates.

◆ ? Null Detection

```
df.isna().any()[df.isna().any()]
```

	0
emp_title	True
emp_length	True
title	True
revol_util	True
mort_acc	True
pub_rec_bankruptcies	True

dtype: bool

```
df.isna().sum().sort_values(ascending=False)
```

	0
mort_acc	37795
emp_title	22927
emp_length	18301
title	1756
pub_rec_bankruptcies	535
revol_util	276
loan_amnt	0
dti	0
application_type	0
initial_list_status	0
total_acc	0
revol_bal	0
pub_rec	0
open_acc	0
earliest_cr_line	0
purpose	0
term	0
loan_status	0
issue_d	0
verification_status	0
annual_inc	0
home_ownership	0
sub_grade	0
grade	0
installment	0
int_rate	0
address	0

dtype: int64

```
def missing_data(df):
    total_missing_df = df.isnull().sum().sort_values(ascending=False)
    percent_missing_df = (df.isnull().sum()/df.isna().count()*100).sort_values(ascending=False)
    missing_data_df = pd.concat([total_missing_df, percent_missing_df], axis=1, keys=['Total', 'Percent'])
    return missing_data_df

missing_pct = missing_data(df)
missing_pct[missing_pct['Total']>0]
```



	Total	Percent
mort_acc	37795	9.543469
emp_title	22927	5.789208
emp_length	18301	4.621115
title	1756	0.443401
pub_rec_bankruptcies	535	0.135091
revol_util	276	0.069692



Insight

Following columns has missing values

1. emp_title has 5.78% missing values
2. emp_length has 4.62% missing values
3. title has 0.44% missing values
4. revol_util has 0.06% missing values
5. mort_acc has 9.54% missing values
6. pub_rec_bankruptcies has 0.13% missing values

Action

- Since ML algorithm do not work on columns which has missing values so we need to impute these missing values.

```
df.isna().sum().sum()
# since there are 81590 rows are null , we cant drop na ...
```

 81590

```
#checking the unique values for columns
for _ in df.columns:
    print()
    print(f'Total Unique Values in {_} column are :- {df[_].nunique()}')
    print(f'Unique Values in {_} column are :-\n {df[_].unique()}')
    print(f'Value_counts of {_} column :-\n {df[_].value_counts()}')
    print()
    print('-'*120)
```



```
Name: count, dtype: int64
```

```
Total Unique Values in address column are :- 393700
Unique Values in address column are :-
['0174 Michelle Gateway\r\nMendozaberg, OK 22690'
'1076 Carney Fort Apt. 347\r\nLoganmouth, SD 05113'
'87025 Mark Dale Apt. 269\r\nNew Sabrina, WV 05113' ...
'953 Matthew Points Suite 414\r\nReedfort, NY 70466'
'7843 Blake Freeway Apt. 229\r\nNew Michael, FL 29597'
'787 Michelle Causeway\r\nBriannaton, AR 48052']
Value_counts of address column :-
address
USCGC Smith\r\nFPO AE 70466      8
USS Johnson\r\nFPO AE 48052      8
USNS Johnson\r\nFPO AE 05113     8
USS Smith\r\nFPO AP 70466        8
USNS Johnson\r\nFPO AP 48052     7
..
455 Tricia Cove\r\nAustinbury, FL 00813      1
7776 Flores Fall\r\nFernandezshire, UT 05113  1
6577 Mia Harbors Apt. 171\r\nRobertshire, OK 22690  1
8141 Cox Greens Suite 186\r\nMadisonstad, VT 05113  1
787 Michelle Causeway\r\nBriannaton, AR 48052  1
Name: count, Length: 393700, dtype: int64
```

✓ Null Treatment:

```
df.loc[df['revol_util'].isna(), 'revol_util'] = 0.0
df.loc[df['mort_acc'].isna(), 'mort_acc'] = 0.0
df.loc[df['pub_rec_bankruptcies'].isna(), 'pub_rec_bankruptcies'] = 0.0
df.loc[df['emp_title'].isna(), 'emp_title'] = 'No Employee Title'
df.loc[df['title'].isna(), 'title'] = 'Unavailable'
df['emp_length'] = df['emp_length'].fillna('< 1 year')
```

```
df.isna().sum()
```



	0
loan_amnt	0
term	0
int_rate	0
installment	0
grade	0
sub_grade	0
emp_title	0
emp_length	0
home_ownership	0
annual_inc	0
verification_status	0
issue_d	0
loan_status	0
purpose	0
title	0
dti	0
earliest_cr_line	0
open_acc	0
pub_rec	0
revol_bal	0
revol_util	0
total_acc	0
initial_list_status	0
application_type	0
mort_acc	0
pub_rec_bankruptcies	0
address	0

dtype: int64

df.describe().T



	count	mean	std	min	25%	50%	75%	max
loan_amnt	396030.0	14113.888089	8357.441341	500.00	8000.00	12000.00	20000.00	40000.00
int_rate	396030.0	13.639400	4.472157	5.32	10.49	13.33	16.49	30.99
installment	396030.0	431.849698	250.727790	16.08	250.33	375.43	567.30	1533.81
annual_inc	396030.0	74203.175798	61637.621158	0.00	45000.00	64000.00	90000.00	8706582.00
dti	396030.0	17.379514	18.019092	0.00	11.28	16.91	22.98	9999.00
open_acc	396030.0	11.311153	5.137649	0.00	8.00	10.00	14.00	90.00
pub_rec	396030.0	0.178191	0.530671	0.00	0.00	0.00	0.00	86.00
revol_bal	396030.0	15844.539853	20591.836109	0.00	6025.00	11181.00	19620.00	1743266.00
revol_util	396030.0	53.754260	24.484857	0.00	35.80	54.80	72.90	892.30
total_acc	396030.0	25.414744	11.886991	2.00	17.00	24.00	32.00	151.00
mort_acc	396030.0	1.640873	2.111249	0.00	0.00	1.00	3.00	34.00
pub_rec_bankruptcies	396030.0	0.121483	0.355962	0.00	0.00	0.00	0.00	8.00



df.describe(include='object').T



	count	unique	top	freq	
term	396030	2	36 months	302005	
grade	396030	7	B	116018	
sub_grade	396030	35	B3	26655	
emp_title	396030	173106	No Employee Title	22927	
emp_length	396030	11	10+ years	126041	
home_ownership	396030	6	MORTGAGE	198348	
verification_status	396030	3	Verified	139563	
issue_d	396030	115	Oct-2014	14846	
loan_status	396030	2	Fully Paid	318357	
purpose	396030	14	debt_consolidation	234507	
title	396030	48817	Debt consolidation	152472	
earliest_cr_line	396030	684	Oct-2000	3017	
initial_list_status	396030	2	f	238066	
application_type	396030	3	INDIVIDUAL	395319	
address	396030	393700	USCGC Smith\r\nFPO AE 70466		8

Feature Engineering

```
df['pub_rec'] = [1 if i > 1 else 0 for i in df['pub_rec']]
df['mort_acc'] = [1 if i > 1 else 0 for i in df['mort_acc']]
df['pub_rec_bankruptcies'] = [1 if i > 1 else 0 for i in df['pub_rec_bankruptcies']]
```

```
df.sample()
```



	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ownership	annual_inc	veri
391047	28500.0	60 months	22.15	789.58	E	E5	Maintenance	10+ years	MORTGAGE	75000.0	

```
#Split issue_date into month and year
df[['issue_month', 'issue_year']] = df['issue_d'].str.split('-', expand=True)
df.drop(['issue_d'], axis=1, inplace=True)
```

```
#Split er_cr_line date into month and year
df[['er_cr_line_m', 'er_cr_line_y']] = df['earliest_cr_line'].str.split('-', expand=True)
df.drop(['earliest_cr_line'], axis=1, inplace=True)
```

```
df['address']
```



	address
0	0174 Michelle Gateway\r\nMendozaberg, OK 22690
1	1076 Carney Fort Apt. 347\r\nLoganmouth, SD 05113
2	87025 Mark Dale Apt. 269\r\nNew Sabrina, WV 05113
3	823 Reid Ford\r\nDelacruzside, MA 00813
4	679 Luna Roads\r\nGreggshire, VA 11650
...	...
396025	12951 Williams Crossing\r\nJohnnyville, DC 30723
396026	0114 Fowler Field Suite 028\r\nRachelborough, ...
396027	953 Matthew Points Suite 414\r\nReedfort, NY 7...
396028	7843 Blake Freeway Apt. 229\r\nNew Michael, FL...
396029	787 Michelle Causeway\r\nBriannaton, AR 48052

396030 rows x 1 columns

dtype: object


```
#Split address into State and Zip code
import re
df[['state','zipcode']] = df['address'].str.extract(r'([A-Z]{2}) (\d{5})')
df.drop(['address'], axis=1, inplace=True)
```

```
df['state'].nunique() , df['zipcode'].nunique()
```

```
(54, 10)
```

```
df['state'].isna().sum() , df['zipcode'].isna().sum()
```

```
(0, 0)
```

```
df['emp_length_yrs'] = df['emp_length'].str.extract('(\d+)')
df.drop(['emp_length'], axis=1, inplace=True)
```

```
df['term'] = df['term'].str.split().str[0].astype('object')
```

```
df.sample()
```

```
(54, 10)
```

	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	home_ownership	annual_inc	verification_status
357028	28000.0	36	6.49	858.05	A	A2	Sr. Manager	MORTGAGE	166000.0	Source Verifie

```
df.shape
```

```
(396030, 30)
```

```
# List of categorical columns
cat_cols = df.select_dtypes(include='object')
```

```
# List of numerical columns
num_cols = df.select_dtypes(exclude='object')
```

```
cat_cols.sample(3)
```

```
(54, 10)
```

	term	grade	sub_grade	emp_title	home_ownership	verification_status	loan_status	purpose	title	in
330489	60	C	C5	ABCUSD	RENT	Verified	Fully Paid	debt_consolidation	Debt consolidation	
63729	36	A	A2	No Employee Title	MORTGAGE	Verified	Fully Paid	debt_consolidation	Debt consolidation	
104692	60	F	F1	Lesley University	RENT	Verified	Charged Off	debt_consolidation	Debt Consolidation	

```
num_cols.sample(3)
```

```
(54, 10)
```

	loan_amnt	int_rate	installment	annual_inc	dti	open_acc	pub_rec	revol_bal	revol_util	total_acc	mort_acc	p
210051	8400.0	16.99	299.45	45000.0	28.80	18.0	0	16309.0	53.0	39.0	0	
10288	10000.0	10.99	327.34	60000.0	21.56	27.0	0	19962.0	41.8	35.0	0	
287962	1400.0	11.99	46.50	61000.0	22.11	12.0	0	19022.0	82.7	24.0	0	

```
num_cols.skew()
```



	0
loan_amnt	0.777285
int_rate	0.420669
installment	0.983598
annual_inc	41.042725
dti	431.051225
open_acc	1.213019
pub_rec	6.812303
revol_bal	11.727515
revol_util	-0.074238
total_acc	0.864328
mort_acc	0.412225
pub_rec_bankruptcies	12.936099

dtype: float64

▼ Insights

- Features are Right skewed

Action

- Need to apply log transformations in order to normalise them

```
df1 = df.copy()
```

```
df1.sample()
```



	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	home_ownership	annual_inc	verification_status
349008	1500.0	36	12.84	50.43	C	C2	Christmas tree shops	RENT	55400.0	Not Verifie

▼ Q1. What percentage of customers have fully paid their Loan Amount?

```
df['loan_status'].value_counts(normalize=True)*100
```



	proportion
loan_status	
Fully Paid	80.387092
Charged Off	19.612908

dtype: float64

▼ Insights:

- Target variable distribution is 80%-20%. Data is **significantly imbalanced**

 Graphical Analysis:

uni / bi / multi variate Analysis

```
cp = ['indigo','m','darkviolet','magenta','mediumorchid','violet','purple','orchid','mediumpurple','deeppink','blueviolet','
```

```
num_cols.iloc[:,[0,2,3,4,5,6,8,9,10]].sample()
```

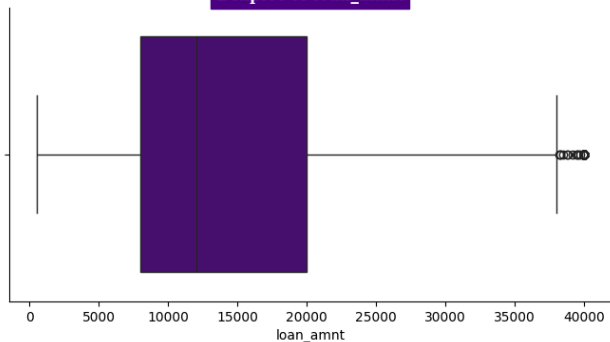


	loan_amnt	installment	annual_inc	dti	open_acc	pub_rec	revol_util	total_acc	mort_acc	
67289	12000.0	245.57	120000.0	17.69	12.0	0	16.9	53.0	1	

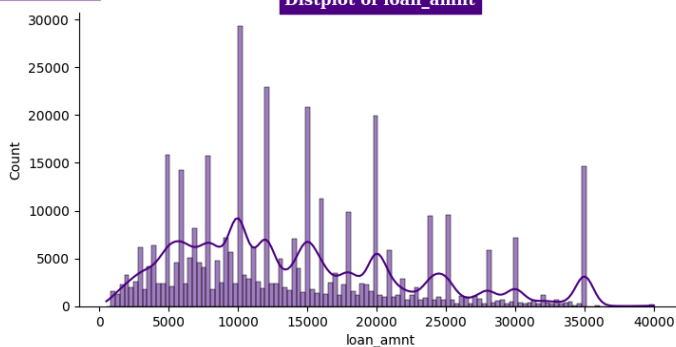
```
plt.style.use('default')
plt.style.use('seaborn-bright')
outlier_graphical_cols = num_cols.iloc[:, [0,2,3,4,5,6,8,9,10]]
for _, col in enumerate(outlier_graphical_cols.columns):
    plt.figure(figsize=(18,4))
    plt.suptitle(f'plot of {col}', fontsize=15, fontfamily='serif', fontweight='bold', backgroundcolor=cp[_], color='w')
    plt.subplot(121)
    sns.boxplot(x=df[col], color=cp[_])
    plt.title(f'Boxplot of {col}', fontsize=12, fontfamily='serif', fontweight='bold', backgroundcolor=cp[_], color='w')
    plt.subplot(122)
    sns.histplot(x=df[col], kde=True, color=cp[_])
    plt.title(f'Distplot of {col}', fontsize=12, fontfamily='serif', fontweight='bold', backgroundcolor=cp[_], color='w')
    sns.despine()
plt.show()
```



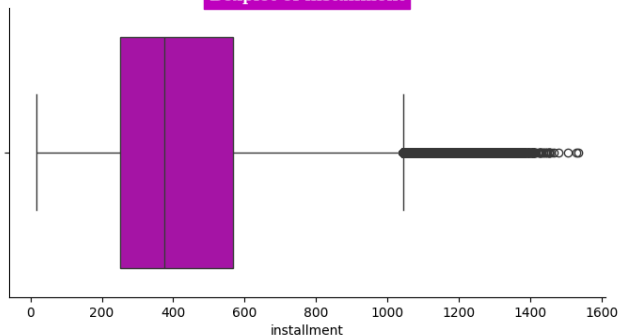
Boxplot of loan_amnt



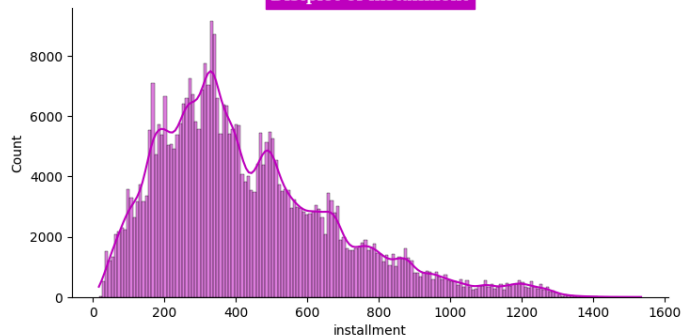
plot of loan_amnt



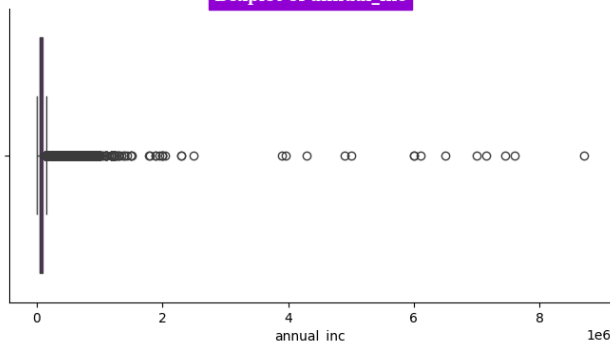
Boxplot of installment



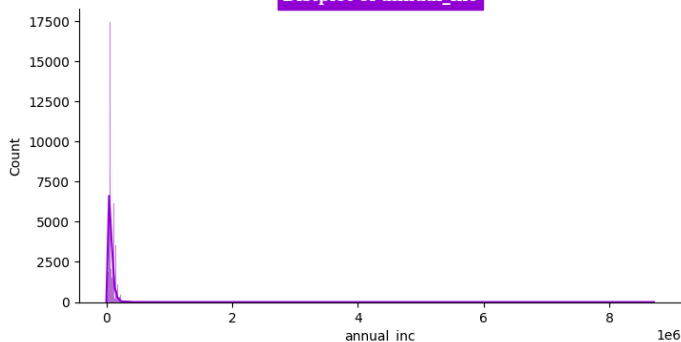
plot of installment



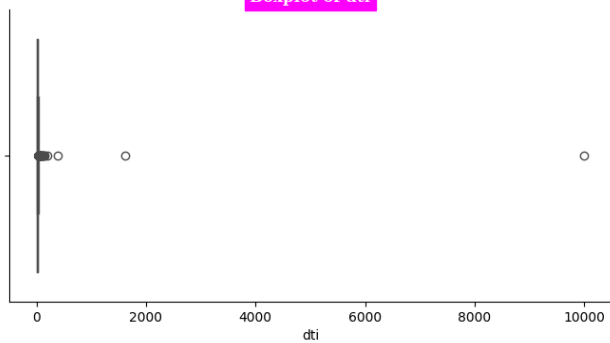
Boxplot of annual_inc



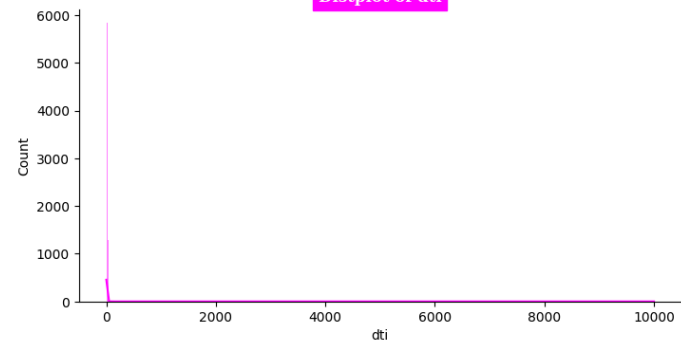
plot of annual_inc



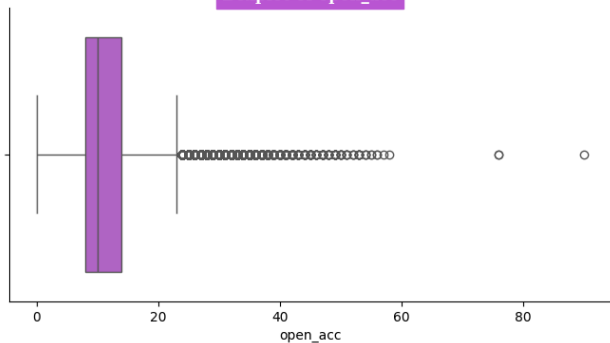
Boxplot of dti



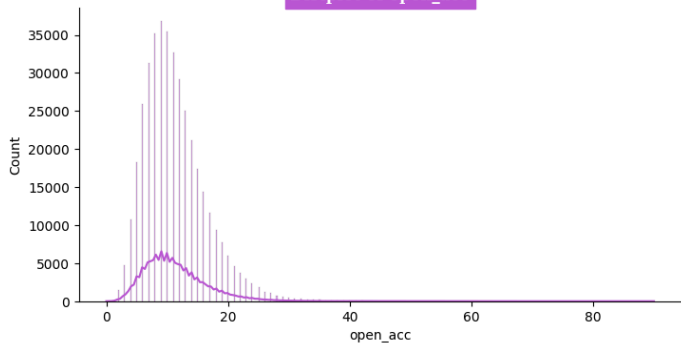
plot of dti



Boxplot of open_acc



plot of open_acc

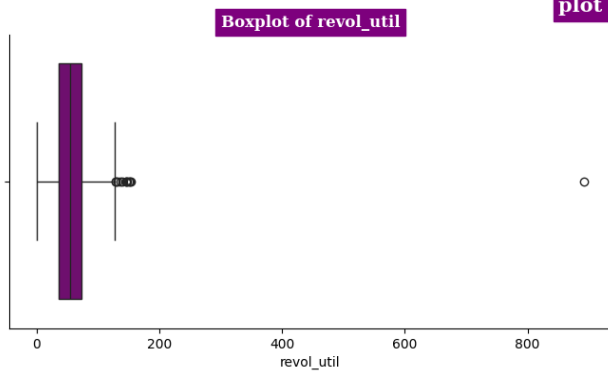
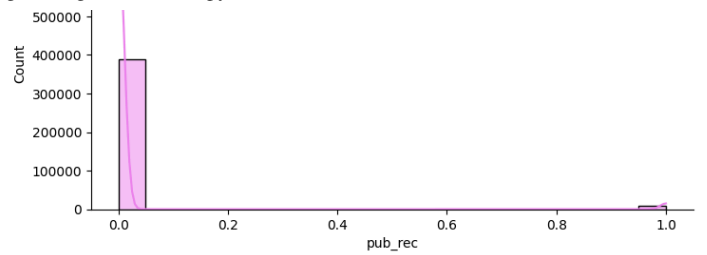
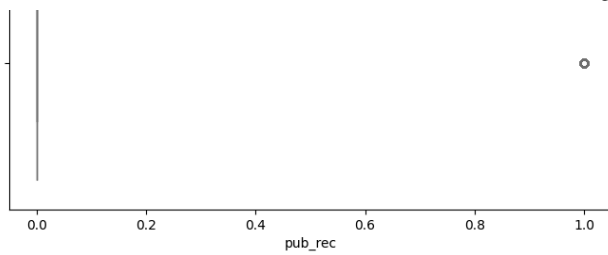


Boxplot of pub_rec

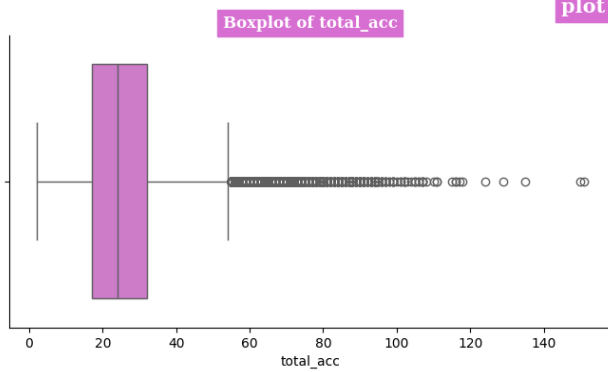
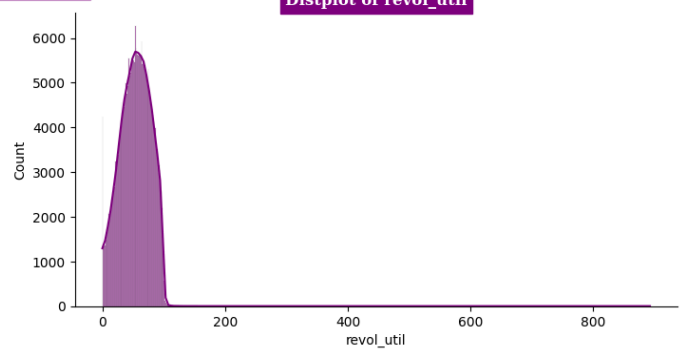


plot of pub_rec

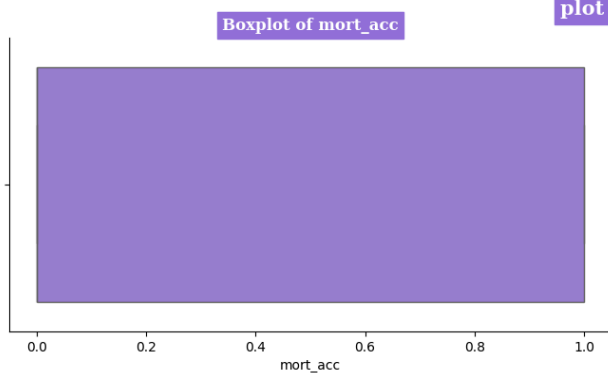
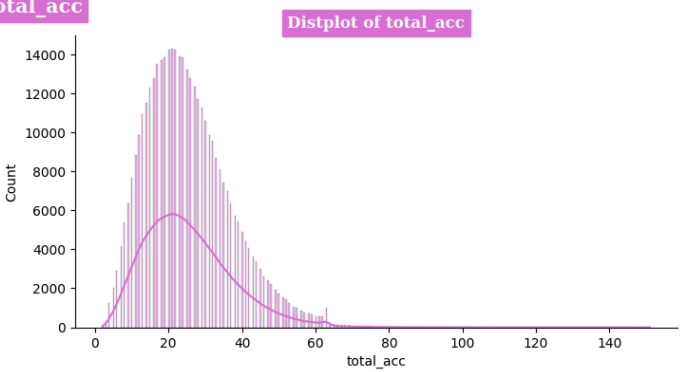




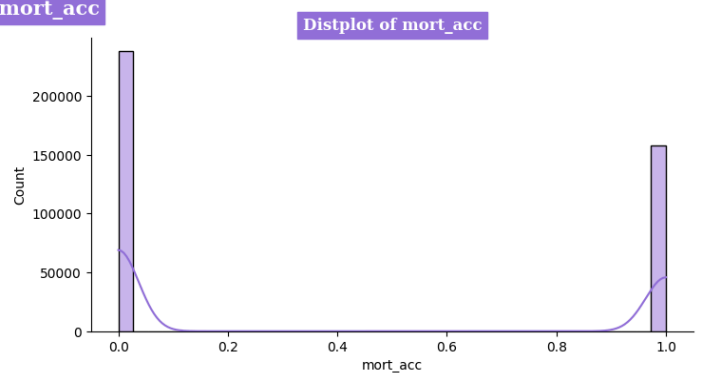
plot of revol_util



plot of total_acc



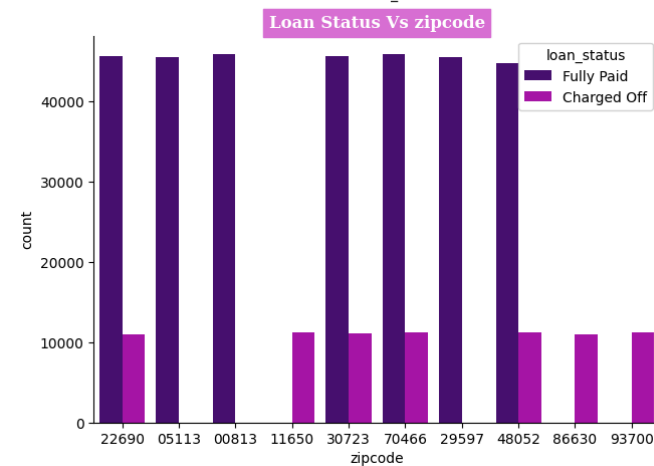
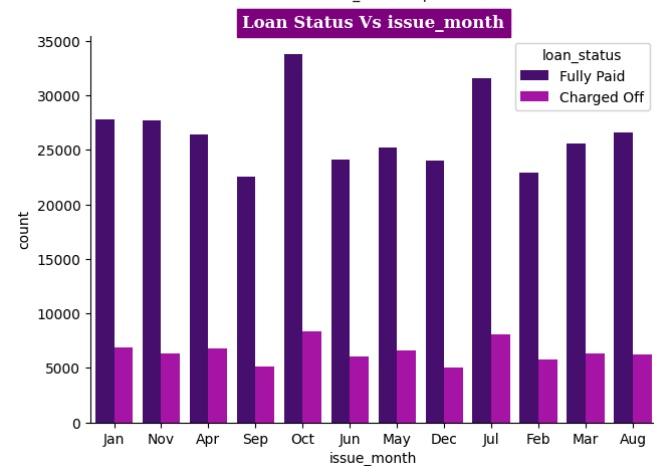
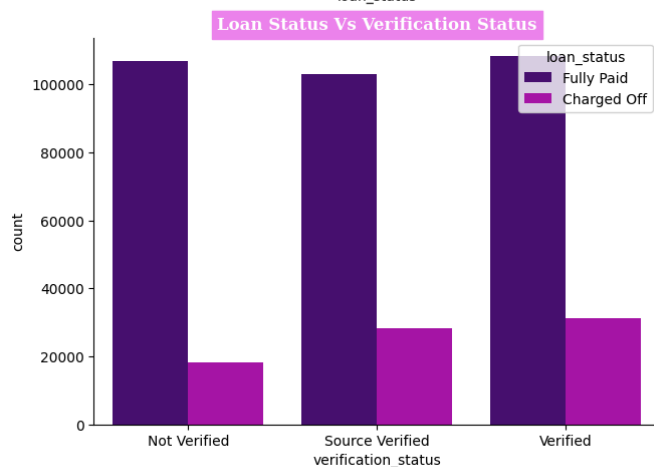
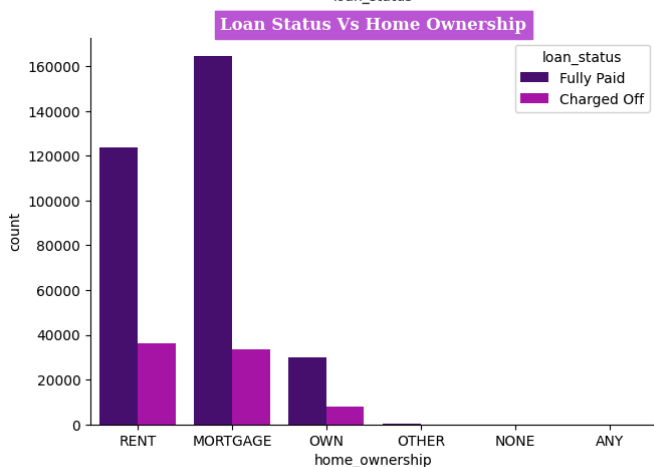
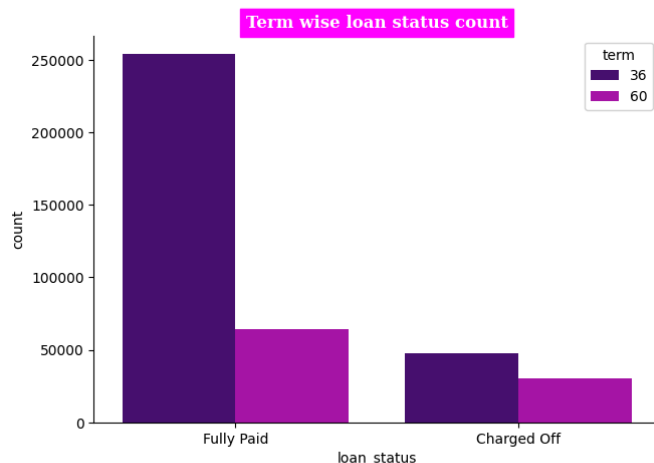
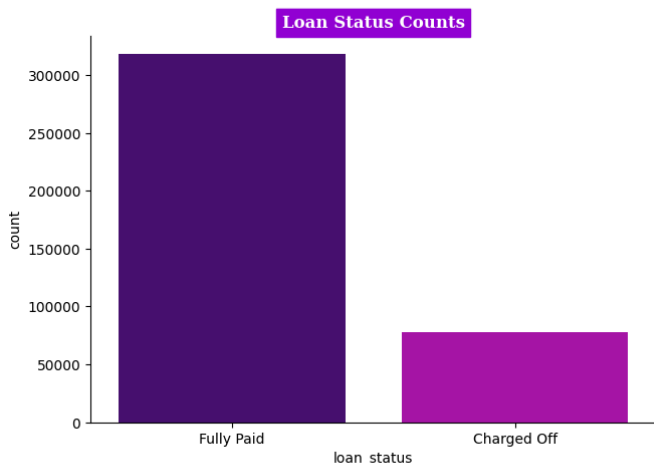
plot of mort_acc



▼ Insights:

1. The analysis suggests a prevalence of outliers, prompting further investigation into outlier detection techniques.
2. Among the numerical features, Potential outliers may still be present.
3. Notably, features such as Pub_rec, Mort_acc, and Pub_rec_bankruptcies display a sparse distribution of unique values, indicating the potential benefit of generating binary features from these variables.

```
#Countplots of various categorical features w.r.t. to target variable loan_status
plt.figure(figsize=(16,17))
plt.suptitle('Countplots of various categorical features w.r.t. to target variable loan_status',
            fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor=cp[1],color='w')
plt.subplot(321)
sns.countplot(data=df, x='loan_status',palette=cp)
plt.title('Loan Status Counts',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[2],color='w')
plt.subplot(322)
sns.countplot(data=df, x='loan_status', hue='term',palette=cp)
plt.title('Term wise loan status count',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[3],color='w')
plt.subplot(323)
sns.countplot(data=df, x='home_ownership', hue='loan_status',palette=cp)
plt.title('Loan Status Vs Home Ownership',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[4],color='w')
plt.subplot(324)
sns.countplot(data=df, x='verification_status', hue='loan_status',palette=cp)
plt.title('Loan Status Vs Verification Status',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[5],color='w')
plt.subplot(325)
sns.countplot(data=df, x='issue_month', hue='loan_status',palette=cp)
plt.title('Loan Status Vs issue_month',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[6],color='w')
plt.subplot(326)
sns.countplot(data=df, x='zipcode', hue='loan_status',palette=cp)
plt.title('Loan Status Vs zipcode',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[7],color='w')
sns.despine()
plt.show()
```

**Countplots of various categorical features w.r.t. to target variable loan_status**


```
zip_codes = ["11650", "86630", "93700"]
states = df[df['zipcode'].isin(zip_codes)][['state']]
```

```
for zip_code, state in zip(zip_codes, states):
    print(f"Zip code: {zip_code}, State: {state}")
```

```
➦ Zip code: 11650, State: VA
    Zip code: 86630, State: MI
    Zip code: 93700, State: MD
```

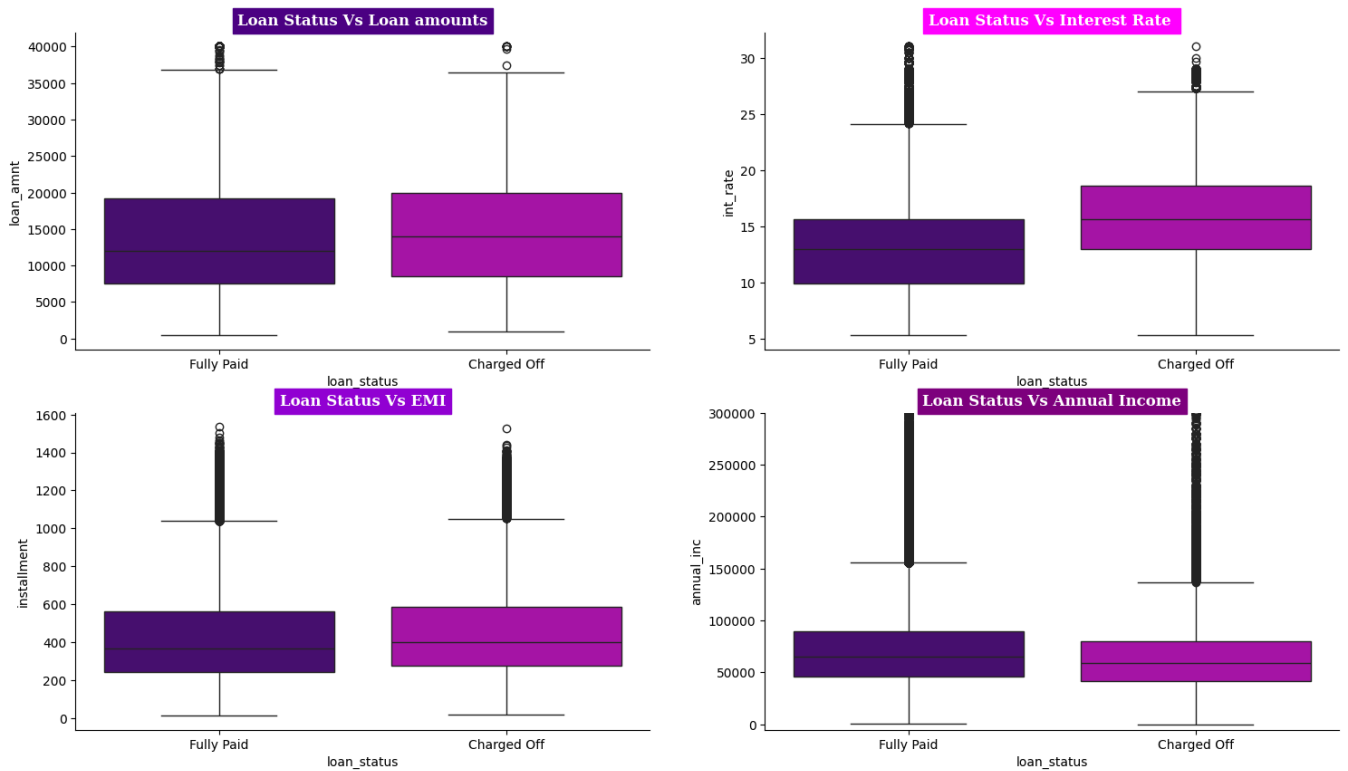
🔍 Observations:

- It's been observed that loans haven't been completely repaid in zip codes 11650, 86630, and 93700.
- Loans haven't been repaid by borrowers residing in 'VA', 'MI', and 'MD'.

```
#Boxplot of various cont. features w.r.t. target variable loan_status
plt.figure(figsize=(18,10))
plt.suptitle('Boxplot of various cont. features w.r.t. target variable loan_status',
            fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor=cp[1],color='w')
plt.subplot(221)
sns.boxplot(data=df, x='loan_status', y='loan_amnt',palette=cp)
plt.title('Loan Status Vs Loan amounts',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[0],color='w')
plt.subplot(222)
sns.boxplot(data=df, x='loan_status', y='int_rate',palette=cp)
plt.title('Loan Status Vs Interest Rate ',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[3],color='w')
plt.subplot(223)
sns.boxplot(data=df, x='loan_status', y='installment',palette=cp)
plt.title('Loan Status Vs EMI',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[2],color='w')
plt.subplot(224)
sns.boxplot(data=df, x='loan_status', y='annual_inc',palette=cp)
plt.ylim(bottom=-5000, top=300000)
plt.title('Loan Status Vs Annual Income',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[6],color='w')
sns.despine()
plt.show()
```



Boxplot of various cont. features w.r.t. target variable loan_status



Observations:

- Charged Off customers exhibit a notably higher median interest rate compared to Fully Paid customers.
- The median annual income of Charged Off customers is lower than that of Fully Paid customers.
- Charged Off customers tend to have a higher median EMI compared to Fully Paid customers.
- The median loan amount for Charged Off customers surpasses that of Fully Paid customers.

```
df.sample()
```



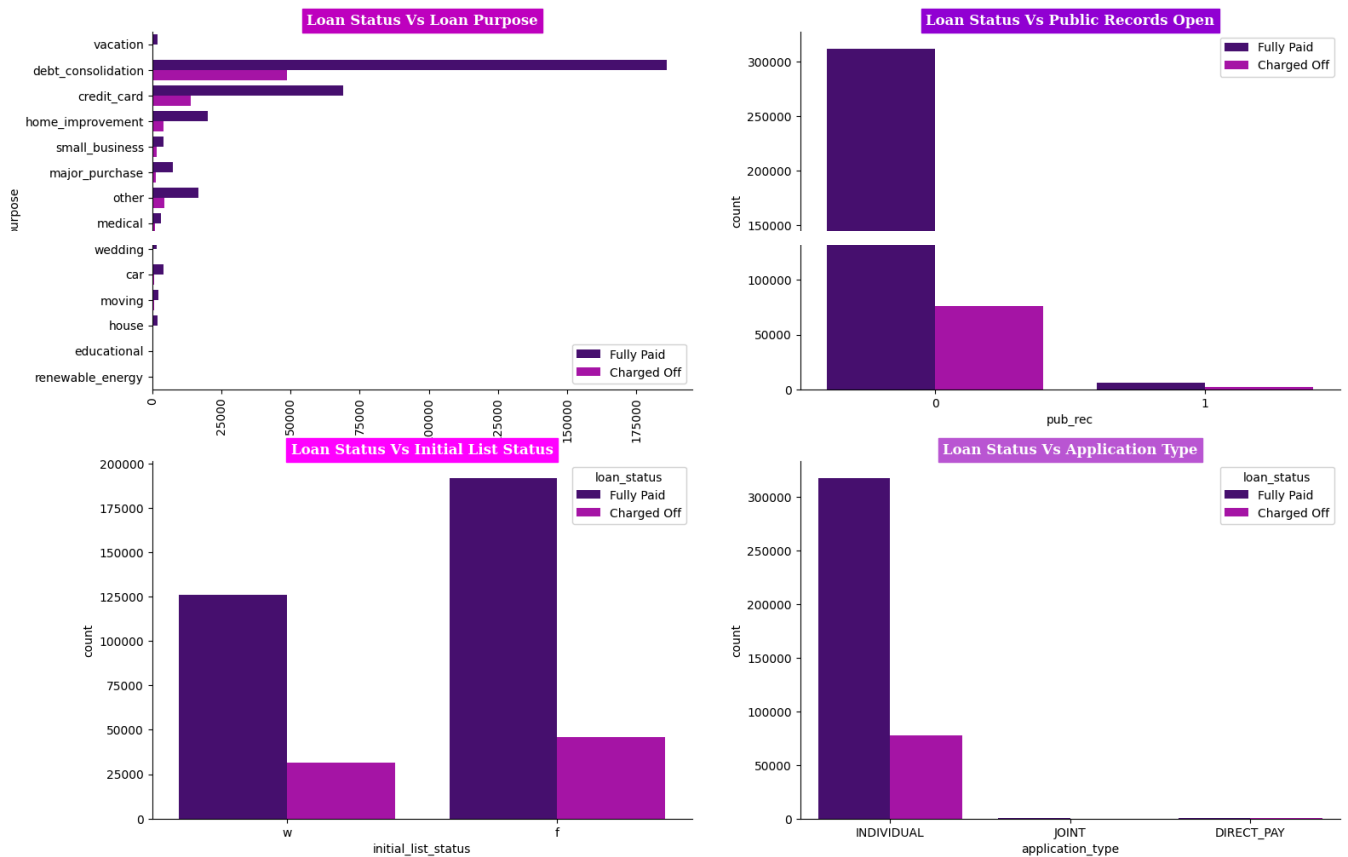
	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	home_ownership	annual_inc	verification_statu
317265	7750.0	36	16.2	273.24	C	C4	WholeFood Market Co	RENT	28000.0	Not Verifie

```
#Countplot of categorical variables w.r.t. target variable loan_status
plt.figure(figsize=(18,12))
plt.suptitle('Countplot of categorical variables w.r.t. target variable loan_status',
            fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor=cp[0],color='w')
plt.subplot(221)
sns.countplot(data=df, y='purpose', hue='loan_status',palette=cp)
plt.xticks(rotation=90)
plt.title('Loan Status Vs Loan Purpose',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[1],color='w')
plt.legend(loc=4)
plt.subplot(222)
sns.countplot(data=df, x='pub_rec',hue='loan_status',palette=cp)
plt.title('Loan Status Vs Public Records Open',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[2],color=
plt.legend(loc=1)
```

```
plt.subplot(223)
sns.countplot(data=df, x='initial_list_status', hue='loan_status', palette=cp)
plt.title('Loan Status Vs Initial List Status', fontsize=12, fontfamily='serif', fontweight='bold', backgroundcolor=cp[3], color=
plt.subplot(224)
sns.countplot(data=df, x='application_type', hue='loan_status', palette=cp)
plt.title('Loan Status Vs Application Type', fontsize=12, fontfamily='serif', fontweight='bold', backgroundcolor=cp[4], color='w'
sns.despine()
plt.show()
```



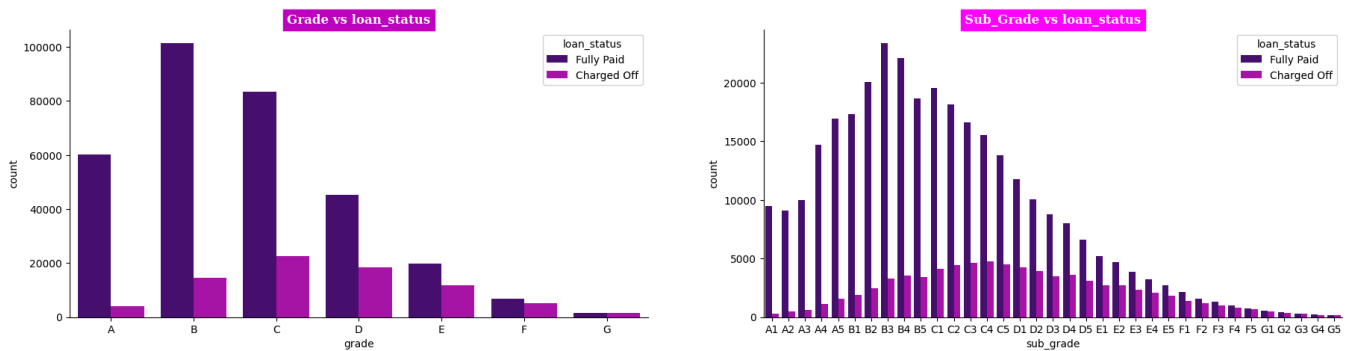
Countplot of categorical variables w.r.t. target variable loan_status



```
plt.figure(figsize=(22,11))
plt.suptitle('Countplot of categorical variables w.r.t. target variable loan_status',
             fontsize=14, fontfamily='serif', fontweight='bold', backgroundcolor=cp[0], color='w')
plt.subplot(221)
grade = sorted(df.grade.unique().tolist())
sns.countplot(x='grade', data=df, hue='loan_status', order=grade, palette=cp)
plt.title('Grade vs loan_status', fontsize=12, fontfamily='serif', fontweight='bold', backgroundcolor=cp[1], color='w')
plt.subplot(222)
sub_grade = sorted(df.sub_grade.unique().tolist())
sns.countplot(x='sub_grade', data=df, hue='loan_status', order=sub_grade, palette=cp)
plt.title('Sub_Grade vs loan_status', fontsize=12, fontfamily='serif', fontweight='bold', backgroundcolor=cp[3], color='w')
sns.despine()
plt.show()
```



Countplot of categorical variables w.r.t. target variable loan_status



Observations:

- Top 2 loan purpose categories are Debit Consolidation and Credit Card
- Topmost loan type application is INDIVIDUAL
- The distribution of open_acc appears to be relatively normal when visualized graphically.
- Charged Off and Fully Paid categories exhibit similar distributions.

```
df.sample()
```



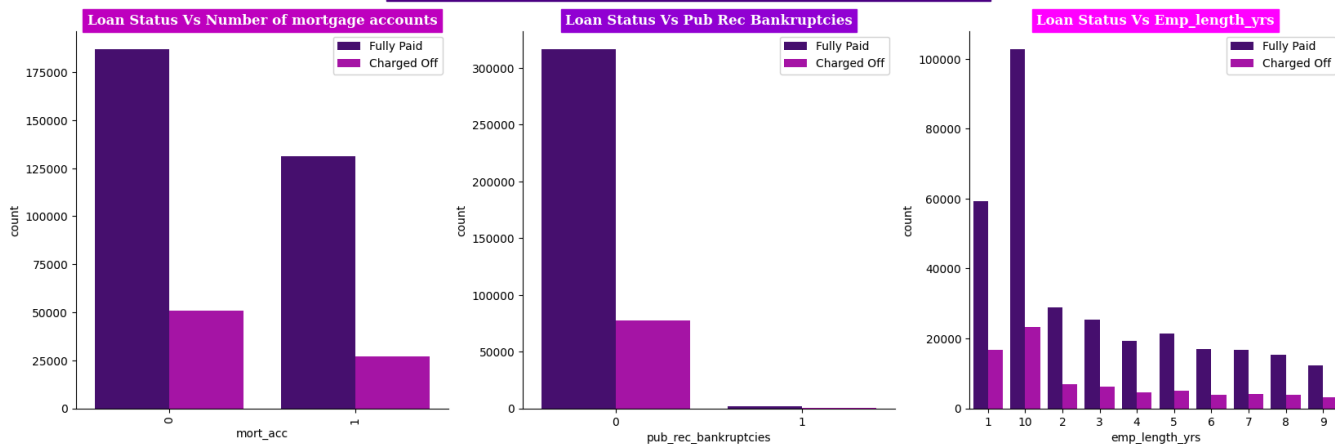
	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	home_ownership	annual_inc	verification_status
21962	8000.0	36	11.44	263.58	B	B4	Branch Manager	RENT	74000.0	Not Verified

```
#Countplot for various categorical features w.r.t. target variable loan_status
```

```
plt.figure(figsize=(20,6))
plt.suptitle('Countplot of categorical variables w.r.t. target variable loan_status',
            fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor=cp[0],color='w')
plt.subplot(131)
sns.countplot(data=df, x='mort_acc',hue='loan_status',palette=cp)
plt.xticks(rotation=90)
plt.title('Loan Status Vs Number of mortgage accounts',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[1]
plt.legend(loc=1)
plt.subplot(132)
sns.countplot(data=df, x='pub_rec_bankruptcies',hue='loan_status',palette=cp)
plt.title('Loan Status Vs Pub Rec Bankruptcies',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[2],color
plt.legend(loc=1)
plt.subplot(133)
order = sorted(df.emp_length_yrs.unique().tolist())
sns.countplot(data=df, x='emp_length_yrs',hue='loan_status',order=order,palette=cp)
plt.title('Loan Status Vs Emp_length_yrs',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor=cp[3],color='w')
plt.legend(loc=1)
sns.despine()
plt.show()
```



Countplot of categorical variables w.r.t. target variable loan_status



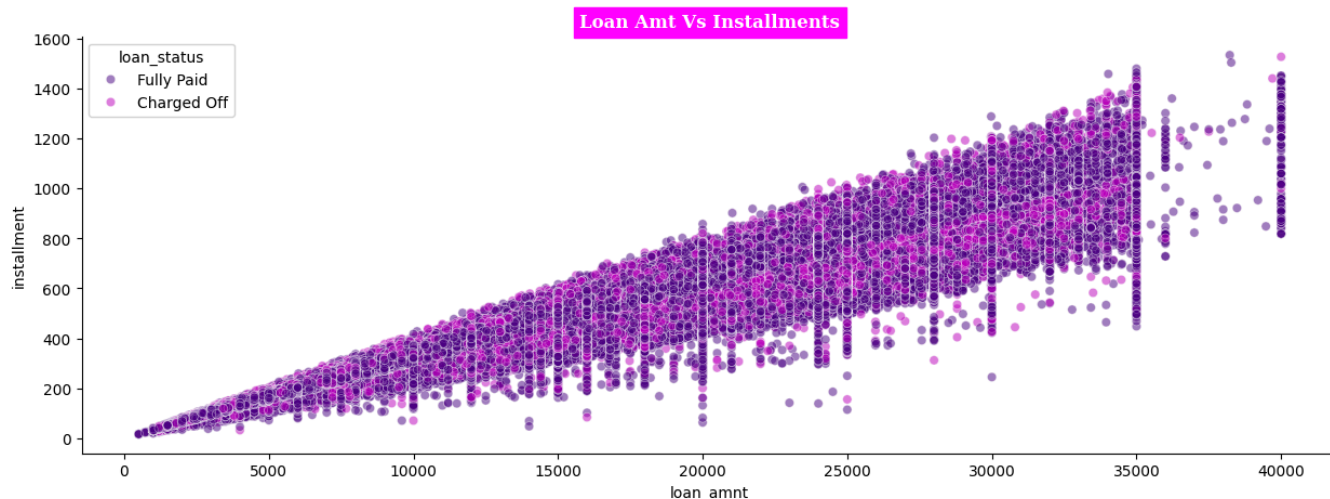
✓ Q2. Comment about the correlation between Loan Amount and Installment features.

```
df[['loan_amnt', 'installment']].corr()
```



	loan_amnt	installment
loan_amnt	1.000000	0.953929
installment	0.953929	1.000000

```
plt.figure(figsize = (15,5))
sns.scatterplot(data = df, x = 'loan_amnt', y = 'installment', alpha = 0.5, hue = 'loan_status', palette = cp)
plt.title('Loan Amt Vs Installments',fontfamily='serif',fontweight='bold',backgroundcolor=cp[3],color='w')
sns.despine()
plt.show()
```



Insights:

The correlation coefficient measures the strength and direction of the linear relationship between two variables. In this case, the correlation coefficient between 'loan_amnt' and 'installment' is quite high, approximately 0.95, indicating a strong positive linear relationship between these

two variables.

- **Loan Terms:** Understanding the relationship between loan amount and installment payments is crucial for setting appropriate loan terms. Lenders can adjust loan terms such as interest rates and repayment periods based on the borrower's ability to handle installment payments associated with different loan amounts.
- **Potential Multicollinearity:** When building predictive models, it's essential to be cautious of multicollinearity between highly correlated predictor variables. Multicollinearity can lead to unstable estimates and difficulties in interpreting the model coefficients. Therefore, it might be necessary to address multicollinearity through techniques such as variable selection or regularization.

✓  Q3. The majority of people have home ownership as ____.

```
(df['home_ownership'].value_counts(normalize=True)*100).to_frame()
```



home_ownership		proportion
MORTGAGE		50.084085
RENT		40.347953
OWN		9.531096
OTHER		0.028281
NONE		0.007828
ANY		0.000758



💡 Insights:

- Mortgage holders comprise the majority with approximately 50.08%, indicating that a significant portion of individuals own homes through Mortgage agreements.
- Renters constitute a substantial portion, accounting for around 40.35% of home ownership types. This suggests a sizable demographic of individuals who opt for renting rather than owning a home.

✓  Q4. People with grades 'A' are more likely to fully pay their loan. (T/F)

```
pd.crosstab(df['grade'],df['loan_status'], normalize = 'index')
```



loan_status	Charged Off	Fully Paid
grade		
A	0.062879	0.937121
B	0.125730	0.874270
C	0.211809	0.788191
D	0.288678	0.711322
E	0.373634	0.626366
F	0.427880	0.572120
G	0.478389	0.521611



💡 Insights:

- True. **Grade 'A' borrowers demonstrate a significantly high likelihood of fully repaying their loans, with approximately 93.71% of loans being fully paid.** This suggests that borrowers with the highest credit rating are more inclined to fulfill their loan obligations successfully.
- The proportion of charged-off loans for grade 'A' borrowers is relatively low, standing at approximately 6.29%. This indicates a low default rate among borrowers with the highest credit rating, emphasizing their creditworthiness and reliability in loan repayment.

✓  Q5. Name the top 2 afforded job titles.

```
df[df['emp_title'] != 'No Employee Title']['emp_title'].value_counts().to_frame().head()
```



emp_title	count
Teacher	4389
Manager	4250
Registered Nurse	1856
RN	1846
Supervisor	1830



```
df.groupby('emp_title')['loan_status'].count().sort_values(ascending=False).to_frame()[1:6]
```



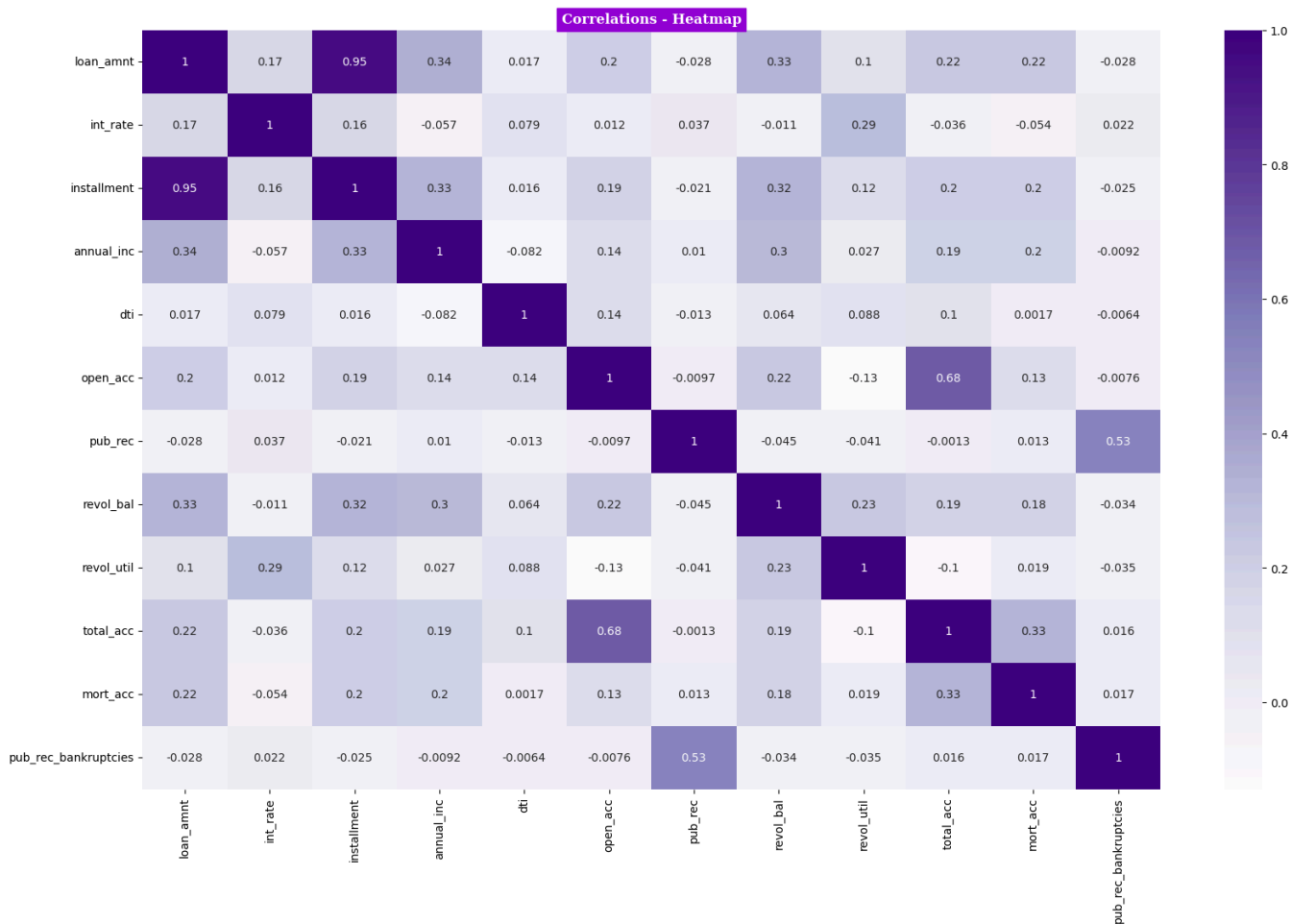
emp_title	loan_status
Teacher	4389
Manager	4250
Registered Nurse	1856
RN	1846
Supervisor	1830



▼ Insights:

- The Most afforded job titles are Teachers & Managers .

```
plt.figure(figsize=(20,12))
sns.heatmap(num_cols.corr(), annot=True, cmap='Purples')
plt.title('Correlations - Heatmap', fontsize=12, fontfamily='serif', fontweight='bold', backgroundcolor=cp[2], color='w')
plt.show()
```



Observations:

- There exists a strong correlation between loan_amnt and installment, indicating that higher loan amounts correspond to larger installment payments.
- The variables total_acc and open_acc exhibit a significant correlation.
- There is a notable correlation between pub_rec_bankruptcies and pub_rec.

Outlier Treatment:

```
numerical_cols = df.select_dtypes(include=np.number).columns
numerical_cols
```

```
Index(['loan_amnt', 'int_rate', 'installment', 'annual_inc', 'dti', 'open_acc',
      'pub_rec', 'revol_bal', 'revol_util', 'total_acc', 'mort_acc',
      'pub_rec_bankruptcies'],
      dtype='object')
```



```
# outlier treatment
def remove_outliers_zscore(df, threshold=2): #(considering 2 std.dev away from mean approx 95% of data)
    """
    Remove outliers from a DataFrame using the Z-score method.

    Parameters:
        df (DataFrame): The input DataFrame.
        threshold (float): The Z-score threshold for identifying outliers.
                           Observations with a Z-score greater than this threshold
                           will be considered as outliers.

    Returns:
        DataFrame: The DataFrame with outliers removed.
    """
    # Calculate Z-scores for numerical columns
    z_scores = (df[numerical_cols] - df[numerical_cols].mean()) / df[numerical_cols].std()

    # Identify outliers
    outliers = np.abs(z_scores) > threshold

    # Keep non-outliers for numerical columns
    df_cleaned = df[~outliers.any(axis=1)]

    return df_cleaned

cleaned_df = remove_outliers_zscore(df1)
print(cleaned_df.shape)
```

↗ (311392, 30)

```
def clip_outliers_zscore(df, threshold=2):
    """
    Clip outliers in a DataFrame using the Z-score method.

    Parameters:
        df (DataFrame): The input DataFrame.
        threshold (float): The Z-score threshold for identifying outliers.
                           Observations with a Z-score greater than this threshold
                           will be considered as outliers.

    Returns:
        DataFrame: The DataFrame with outliers clipped.
    """
    # Calculate Z-scores for numerical columns
    z_scores = (df[numerical_cols] - df[numerical_cols].mean()) / df[numerical_cols].std()

    # Clip outliers
    clipped_values = df[numerical_cols].clip(df[numerical_cols].mean() - threshold * df[numerical_cols].std(),
                                              df[numerical_cols].mean() + threshold * df[numerical_cols].std(),
                                              axis=1)

    # Assign clipped values to original DataFrame
    df_clipped = df.copy()
    df_clipped[numerical_cols] = clipped_values

    return df_clipped
```

```
clipped_df = clip_outliers_zscore(df1)
print(clipped_df.shape)
```

↗ (396030, 30)

```
data = cleaned_df.copy()
cp_data = clipped_df.copy()
data.sample()
```

↗

	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	home_ownership	annual_inc	verification_statu
300484	22000.0	36	9.75	707.3	B	B3	brewer	MORTGAGE	112000.0	Not Verifie

```
data['pub_rec_bankruptcies'].value_counts() , data['pub_rec'].value_counts()
```

↗

```
(pub_rec_bankruptcies
0    311392
Name: count, dtype: int64,
pub_rec
0    311392
Name: count, dtype: int64)
```

```
cp_data['pub_rec_bankruptcies'].value_counts() , cp_data['pub_rec'].value_counts()
```

```
(pub_rec_bankruptcies
0.000000    393705
0.158662     2325
Name: count, dtype: int64,
pub_rec
0.000000    388011
0.301947     8019
Name: count, dtype: int64)
```

```
data.shape
```

```
(311392, 30)
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 311392 entries, 0 to 396029
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   loan_amnt                             311392 non-null float64
1   term                                  311392 non-null object
2   int_rate                              311392 non-null float64
3   installment                           311392 non-null float64
4   grade                                 311392 non-null object
5   sub_grade                             311392 non-null object
6   emp_title                             311392 non-null object
7   home_ownership                        311392 non-null object
8   annual_inc                            311392 non-null float64
9   verification_status                   311392 non-null object
10  loan_status                            311392 non-null object
11  purpose                                311392 non-null object
12  title                                  311392 non-null object
13  dti                                     311392 non-null float64
14  open_acc                               311392 non-null float64
15  pub_rec                                311392 non-null int64
16  revol_bal                              311392 non-null float64
17  revol_util                             311392 non-null float64
18  total_acc                              311392 non-null float64
19  initial_list_status                    311392 non-null object
20  application_type                       311392 non-null object
21  mort_acc                               311392 non-null int64
22  pub_rec_bankruptcies                   311392 non-null int64
23  issue_month                            311392 non-null object
24  issue_year                             311392 non-null object
25  er_cr_line_m                           311392 non-null object
26  er_cr_line_y                           311392 non-null object
27  state                                  311392 non-null object
28  zipcode                                311392 non-null object
29  emp_length_yrs                         311392 non-null object
dtypes: float64(9), int64(3), object(18)
memory usage: 73.6+ MB
```

Manual encoding:

```
data['loan_status']=data.loan_status.map({'Fully Paid':1, 'Charged Off':0})
```

```
data['initial_list_status']=data.initial_list_status.map({'w':0, 'f':1})
```

```
data.head()
```

```
(
  loan_amnt  term  int_rate  installment  grade  sub_grade  emp_title  home_ownership  annual_inc  verification_status  l
0    10000.0   36    11.44      329.48     B      B4      Marketing           RENT      117000.0      Not Verified
1     8000.0   36    11.99      265.68     B      B5      Credit analyst      MORTGAGE      65000.0      Not Verified
2    15600.0   36    10.49      506.97     B      B3      Statistician      RENT      43057.0      Source Verified
3     7200.0   36     6.49      220.65     A      A2      Client Advocate      RENT      54000.0      Not Verified
4    24375.0   60    17.27      609.33     C      C5      Destiny Management Inc.      MORTGAGE      55000.0      Verified
```

✓ Feature selection - done by hypothesis testing & VIF(multicollinearity)

Find VIF after modelling and remove features with high VIF (>5):

```
def calc_vif(X):
    # Calculating the VIF
    vif=pd.DataFrame()
    vif['Feature']=X.columns
    vif['VIF']=[variance_inflation_factor(X.values,i) for i in range(X.shape[1])]
    vif['VIF']=round(vif['VIF'],2)
    vif=vif.sort_values(by='VIF',ascending=False)
    return vif

cat_cols = data.select_dtypes(include=['object']).columns.tolist()
for col in cat_cols:
    chi2, p, dof, expected = chi2_contingency(pd.crosstab(data[col], data['loan_status']))
    if p > 0.05:
        print('>>>>>> Independent feature - Not Significant:',col,' >> p value:',p)

>>>>>> Independent feature - Not Significant: emp_title >> p value: 0.5367121560200798
>>>>>> Independent feature - Not Significant: title >> p value: 1.0
>>>>>> Independent feature - Not Significant: er_cr_line_m >> p value: 0.2722117086158036
>>>>>> Independent feature - Not Significant: state >> p value: 0.76047808977373

## dropping cols based on correlation(heatmap,hypothesis testing)
lt = data.drop(columns=['emp_title','title','sub_grade','er_cr_line_m','er_cr_line_y','initial_list_status',
                        'state','issue_month','issue_year','pub_rec','pub_rec_bankruptcies'],axis=1)

lt.shape

(311392, 19)

lt.sample()

      loan_amnt  term  int_rate  installment  grade  home_ownership  annual_inc  verification_status  loan_status
133540    12000.0   36      9.99        387.15      B      MORTGAGE      80000.0      Source Verified      1  debt_co

#### Performing OneHotEncoding on feature having multiple variable
dummies=['zipcode', 'grade','purpose','home_ownership','verification_status','application_type']
ltd = pd.get_dummies(lt, columns=dummies, drop_first=True)*1

ltd.shape

(311392, 50)

ltd.dtypes
```



0

loan_amnt	float64
term	object
int_rate	float64
installment	float64
annual_inc	float64
loan_status	int64
dti	float64
open_acc	float64
revol_bal	float64
revol_util	float64
total_acc	float64
mort_acc	int64
emp_length_yrs	object
zipcode_05113	int64
zipcode_11650	int64
zipcode_22690	int64
zipcode_29597	int64
zipcode_30723	int64
zipcode_48052	int64
zipcode_70466	int64
zipcode_86630	int64
zipcode_93700	int64
grade_B	int64
grade_C	int64
grade_D	int64
grade_E	int64
grade_F	int64
grade_G	int64
purpose_credit_card	int64
purpose_debt_consolidation	int64
purpose_educational	int64
purpose_home_improvement	int64
purpose_house	int64
purpose_major_purchase	int64
purpose_medical	int64
purpose_moving	int64
purpose_other	int64
purpose_renewable_energy	int64
purpose_small_business	int64
purpose_vacation	int64
purpose_wedding	int64
home_ownership_MORTGAGE	int64
home_ownership_NONE	int64
home_ownership_OTHER	int64
home_ownership_OWN	int64
home_ownership_RENT	int64
verification_status_Source Verified	int64
verification_status_Verified	int64
application_type_INDIVIDUAL	int64

application_type_JOINT

int64

dtype: object

ltd.sample(8)

	loan_amnt	term	int_rate	installment	annual_inc	loan_status	dti	open_acc	revol_bal	revol_util	total_acc	r
93805	5000.0	36	5.32	150.58	35000.0	1	4.12	10.0	2948.0	10.5	13.0	
168139	3000.0	36	13.66	102.04	120000.0	1	21.66	11.0	15633.0	52.1	15.0	
55991	10000.0	36	15.27	347.98	65000.0	1	9.99	13.0	17038.0	94.7	35.0	
212293	21000.0	60	19.99	556.26	74000.0	1	14.47	8.0	1159.0	34.1	18.0	
119035	15600.0	60	13.33	357.59	60000.0	1	6.50	7.0	8162.0	58.7	18.0	
97717	18000.0	60	17.27	449.97	55000.0	0	34.80	11.0	28529.0	88.0	16.0	
150092	20000.0	60	19.52	524.56	94551.0	0	18.44	16.0	13271.0	79.5	34.0	
317966	15000.0	36	11.71	496.14	68400.0	1	5.70	5.0	9368.0	82.2	9.0	

Model:

```
#Prepare X and y dataset i.e. independent and dependent datasets
```

```
X = ltd.drop(['loan_status'], axis=1)
y = ltd['loan_status']
```

```
#Split the data into train ,validation and test
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(249113, 49)
(62279, 49)
(249113,)
(62279,)
```

Minmax scaling the data

```
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_train = pd.DataFrame(X_train, columns=X.columns)
X_test = pd.DataFrame(X_test, columns=X.columns)
```

```
X_train.head()
```

	loan_amnt	term	int_rate	installment	annual_inc	dti	open_acc	revol_bal	revol_util	total_acc	mort_acc	emp_li
0	0.379538	0.0	0.339161	0.411590	0.207250	0.465341	0.368421	0.171897	0.419816	0.276596	0.0	
1	0.643564	1.0	0.680070	0.524221	0.367868	0.252652	0.473684	0.221905	0.590398	0.340426	0.0	
2	0.168317	0.0	0.208625	0.176198	0.134712	0.357576	0.368421	0.052236	0.304392	0.212766	0.0	
3	0.379538	1.0	0.680070	0.307444	0.367868	0.449242	0.315789	0.255109	0.767109	0.297872	1.0	
4	0.368812	0.0	0.543706	0.421460	0.246109	0.315530	0.263158	0.090649	0.614913	0.361702	0.0	

Model-1

```
#Fit the Model on training data
logreg_model = LogisticRegression()
logreg_model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression()
```

```
#Predict the data on test dataset
y_train_pred = logreg_model.predict(X_train)
y_test_pred = logreg_model.predict(X_test)

logreg_model.score(X_test, y_test) , logreg_model.score(X_test, y_test_pred)

(0.8934793429566948, 1.0)
```

If `logreg_model.score(X_test, y_test)` consistently returns 1, it would imply that your model is predicting the test set perfectly, which could be a sign of overfitting, data leakage, or an issue with the evaluation process.

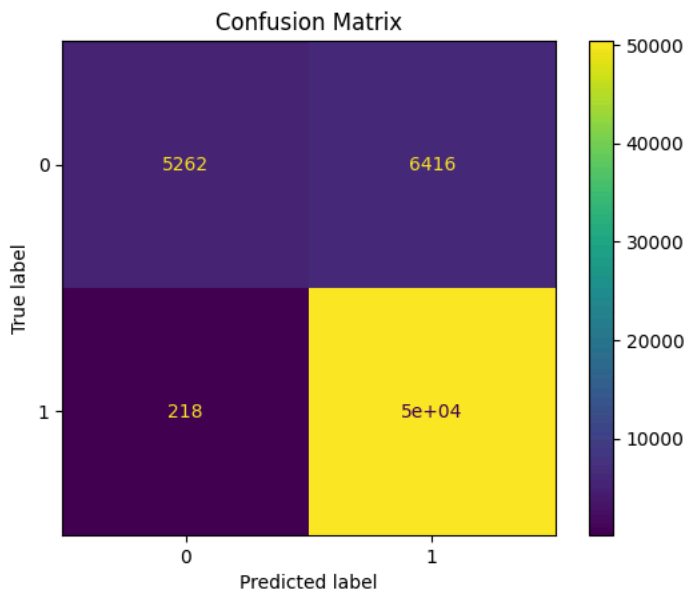
```
#Model Evaluation
print('Train Accuracy :', logreg_model.score(X_train, y_train))
print('Train F1 Score:', f1_score(y_train, y_train_pred))
print('Train Recall Score:', recall_score(y_train, y_train_pred))
print('Train Precision Score:', precision_score(y_train, y_train_pred))

print('\nTest Accuracy :', logreg_model.score(X_test, y_test))
print('Test F1 Score:', f1_score(y_test, y_test_pred))
print('Test Recall Score:', recall_score(y_test, y_test_pred))
print('Test Precision Score:', precision_score(y_test, y_test_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot()
plt.title('Confusion Matrix')
plt.show()
```

```
Train Accuracy : 0.8934339034895811
Train F1 Score: 0.9382212696440165
Train Recall Score: 0.99595357730446
Train Precision Score: 0.886815362280586
```

```
Test Accuracy : 0.8934793429566948
Test F1 Score: 0.9382309124767225
Test Recall Score: 0.995691784747337
Test Precision Score: 0.8870402647933943
```



```
print(classification_report(y_test, y_test_pred))
```

```
precision    recall  f1-score   support

0           0.96      0.45      0.61      11678
1           0.89      1.00      0.94      50601

accuracy          0.89      62279
macro avg         0.92      0.72      0.78      62279
weighted avg      0.90      0.89      0.88      62279
```

- Here the recall value for the 'charged off' is very low, Hence will build a better model

Model-2

Oversampling to balance the target variable

```
sm=SMOTE(random_state=42)
X_train_res, y_train_res = sm.fit_resample(X_train,y_train.ravel())

print(f"Before OverSampling, count of label 1: {sum(y_train == 1)}")
print(f"Before OverSampling, count of label 0: {sum(y_train == 0)}")
print(f"After OverSampling, count of label 1: {sum(y_train_res == 1)}")
print(f"After OverSampling, count of label 0: {sum(y_train_res == 0)}")
```

```
↗ Before OverSampling, count of label 1: 202401
Before OverSampling, count of label 0: 46712
After OverSampling, count of label 1: 202401
After OverSampling, count of label 0: 202401
```

```
model = LogisticRegression()
model.fit(X_train_res, y_train_res)
train_preds = model.predict(X_train)
test_preds = model.predict(X_test)
```

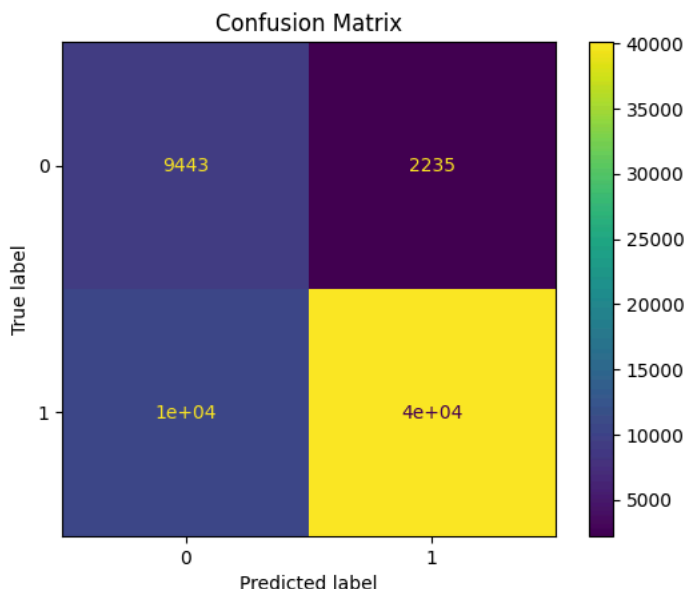
```
#Model Evaluation
print('Train Accuracy :', model.score(X_train, y_train))
print('Train F1 Score:', f1_score(y_train,train_preds))
print('Train Recall Score:', recall_score(y_train,train_preds))
print('Train Precision Score:', precision_score(y_train,train_preds))
```

```
print('\nTest Accuracy :', model.score(X_test,y_test))
print('Test F1 Score:', f1_score(y_test,test_preds))
print('Test Recall Score:', recall_score(y_test,test_preds))
print('Test Precision Score:', precision_score(y_test,test_preds))
```

```
# Confusion Matrix
cm = confusion_matrix(y_test, test_preds)
disp = ConfusionMatrixDisplay(cm)
disp.plot()
plt.title('Confusion Matrix')
plt.show()
```

```
↗ Train Accuracy : 0.7935033498853934
Train F1 Score: 0.8614290524614452
Train Recall Score: 0.7899763341090212
Train Precision Score: 0.9470928304032602
```

```
Test Accuracy : 0.7955330047046356
Test F1 Score: 0.862983924767049
Test Recall Score: 0.7925139819371159
Test Precision Score: 0.9472092968325578
```



```
y_pred = test_preds
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.47	0.81	0.60	11678
1	0.95	0.79	0.86	50601
accuracy			0.80	62279
macro avg	0.71	0.80	0.73	62279
weighted avg	0.86	0.80	0.81	62279

Observations:

- The model demonstrates a high recall score, successfully identifying 80% of actual defaulters.
- However, the precision for the positive class (defaulters) is low; only 47% of predicted defaulters are actually defaulters.
- This high recall and low precision indicate that while the model is effective at flagging most defaulters, it also results in many false positives. Consequently, many deserving customers may be denied loans.
- The low precision adversely affects the F1 score, reducing it to 60%, despite an overall accuracy of 80%. This highlights the trade-off between precision and recall in the model's performance.

Explanation :

- The model is good at catching most people who don't pay back their loans it catches 80% of them.
- But, when it says someone won't pay back, it's right only half of the time. 47% So, there's a chance it's making mistakes and wrongly flagging people.
- Because of these mistakes, some people who deserve loans might not get them.
- Even though the model seems okay overall, its balance between being right and not making mistakes isn't great. It's like a seesaw; when one side goes up, the other goes down.

Regularization Model

#Try with different regularization factor lamda and choose the best to build the model

```
lamb = np.arange(0.01, 100, 10)
```

```
train_scores = []
test_scores = []
```

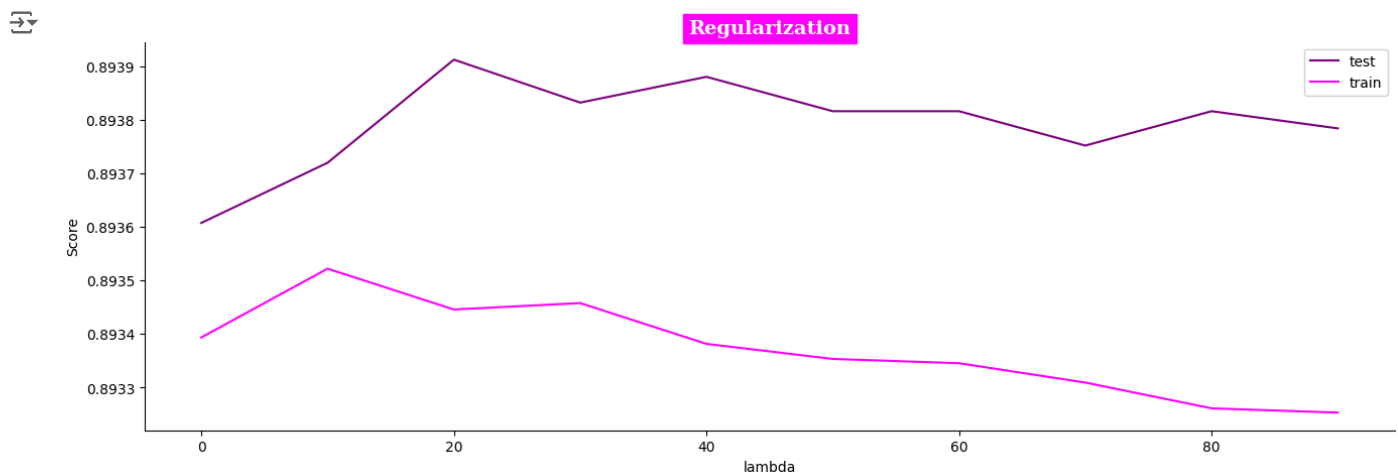
```
for lam in lamb:
    model = LogisticRegression(C = 1/lam)
    model.fit(X_train, y_train)

    tr_score = model.score(X_train, y_train)
    te_score = model.score(X_test, y_test)

    train_scores.append(tr_score)
    test_scores.append(te_score)
```

#Plot the train and test scores with respect lambda values i.e. regularization factors

```
ran = np.arange(0.01, 100, 10)
plt.figure(figsize=(16,5))
sns.lineplot(x=ran,y=test_scores,color='purple',label='test')
sns.lineplot(x=ran,y=train_scores,color='magenta',label='train')
plt.title('Regularization', fontsize=14, fontfamily='serif', fontweight='bold', backgroundcolor='magenta', color='w')
plt.xlabel("lambda")
plt.ylabel("Score")
sns.despine()
plt.show()
```

```
#Check the index of best test score and the check the best test score
```

```
print(np.argmax(test_scores))
print(test_scores[np.argmax(test_scores)])
```

```
2
0.8939128759292859
```

```
#Calculate the best lambda value based on the index of best test score
```

```
best_lamb = 0.01 + (10*2)
best_lamb
```

```
20.01
```

```
#Fit the model using best lambda
```

```
reg_model = LogisticRegression(C=1/best_lamb)
reg_model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(C=0.04997501249375312)
```

```
#Predict the y_values and y_probability values
```

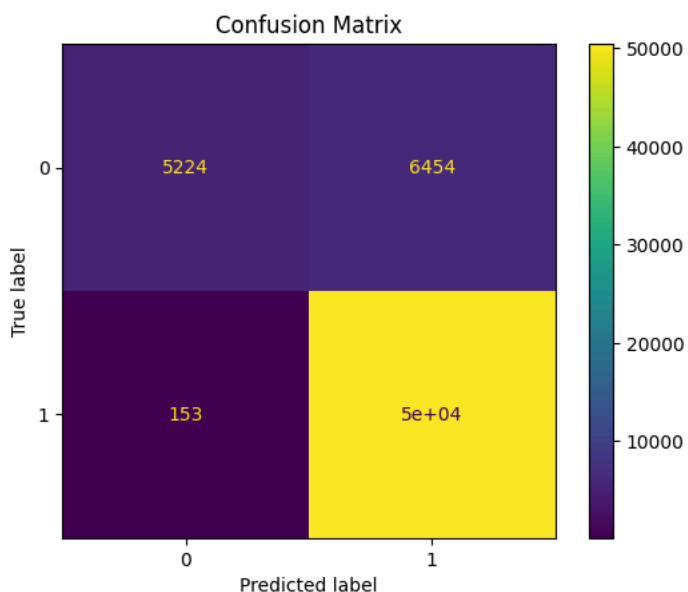
```
y_reg_pred = reg_model.predict(X_test)
y_reg_pred_proba = reg_model.predict_proba(X_test)
```

```
#Print model score
```

```
print(f'Logistic Regression Model Score with best lambda: ',end='')
print(round(model.score(X_test, y_test)*100,2),'%')
```

```
Logistic Regression Model Score with best lambda: 89.38 %
```

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_reg_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot()
plt.title('Confusion Matrix')
plt.show()
```



```
print(classification_report(y_test, y_reg_pred))
```



	precision	recall	f1-score	support
0	0.97	0.45	0.61	11678
1	0.89	1.00	0.94	50601
accuracy			0.89	62279
macro avg	0.93	0.72	0.78	62279
weighted avg	0.90	0.89	0.88	62279

🔍 Observations from classification report:

Regularized model

- Precision : 89%
- Recall : 100%
- F1-score : 94%
- Accuracy : 89%

◆ K-fold - Cross_validation

- cross validation accuracy has to be approx 89%

```
x=scaler.fit_transform(X)
```

```
kfold = KFold(n_splits=10)
```

```
accuracy = np.mean(cross_val_score(reg_model,x,y,cv=kfold,scoring='accuracy'))
```

```
print("Cross Validation accuracy : {:.3f}".format(accuracy))
```



Cross Validation accuracy : 0.894

```
cm = confusion_matrix(y_test, y_reg_pred)
```

```
cm_df = pd.DataFrame(cm, index=['Defaulter','Fully paid'], columns=['Defaulter','Fully paid'])
```

```
cm_df
```



	Defaulter	Fully paid
Defaulter	5224	6454
Fully paid	153	50448

Next steps:

[Generate code with cm_df](#)
[View recommended plots](#)
[New interactive sheet](#)

🔍 Insights:

- TN = 5223 (True Negative: Correctly predicted Charged Off)
- TP = 50450 (True Positive: Correctly predicted Fully Paid)

- FP = 6455 (False Positive: Predicted Fully Paid but actually Charged Off)
- FN = 151 (False Negative: Predicted Charged Off but actually Fully Paid)
- Actual Negative (Charged Off) = 5223 + 6455 = 11678
- Actual Positive (Fully Paid) = 151 + 50450 = 50601
- Predicted Negative (Charged Off) = 5223 + 151 = 5374
- Predicted Positive (Fully Paid) = 6455 + 50450 = 56905

```
#Collect the model coefficients and print those in dataframe format
coeff_df = pd.DataFrame()
coeff_df['Features'] = X_train_res.columns
coeff_df['Weights'] = model.coef_[0]
coeff_df['ABS_Weights'] = abs(coeff_df['Weights'])
coeff_df = coeff_df.sort_values(['ABS_Weights'], ascending=False)
coeff_df
```



	Features	Weights	ABS_Weights	
13	zipcode_11650	-5.328726	5.328726	 
20	zipcode_93700	-5.325475	5.325475	
19	zipcode_86630	-5.304045	5.304045	
12	zipcode_05113	2.176578	2.176578	
15	zipcode_29597	2.168741	2.168741	
17	zipcode_48052	-1.387432	1.387432	
16	zipcode_30723	-1.346745	1.346745	
18	zipcode_70466	-1.337402	1.337402	
14	zipcode_22690	-1.330057	1.330057	
5	dti	-1.009213	1.009213	
4	annual_inc	0.981541	0.981541	
24	grade_E	-0.832342	0.832342	
23	grade_D	-0.714739	0.714739	
2	int_rate	-0.668769	0.668769	
22	grade_C	-0.558078	0.558078	
1	term	-0.473640	0.473640	
6	open_acc	-0.405000	0.405000	
25	grade_F	-0.378117	0.378117	
37	purpose_small_business	-0.368059	0.368059	
9	total_acc	0.334106	0.334106	
8	revol_util	-0.307717	0.307717	
48	application_type_JOINT	0.254217	0.254217	
39	purpose_wedding	0.233529	0.233529	
21	grade_B	-0.225131	0.225131	
40	home_ownership_MORTGAGE	0.212237	0.212237	
3	installment	-0.201611	0.201611	
45	verification_status_Source Verified	-0.201267	0.201267	
7	revol_bal	0.200189	0.200189	
47	application_type_INDIVIDUAL	-0.169025	0.169025	
0	loan_amnt	-0.148693	0.148693	
11	emp_length_yrs	0.117352	0.117352	
46	verification_status_Verified	-0.115029	0.115029	
30	purpose_home_improvement	-0.109624	0.109624	
33	purpose_medical	-0.095869	0.095869	
43	home_ownership_OWEN	0.082271	0.082271	
34	purpose_moving	-0.073242	0.073242	
26	grade_G	-0.056561	0.056561	
38	purpose_vacation	-0.054348	0.054348	
35	purpose_other	-0.053552	0.053552	
10	mort_acc	0.050138	0.050138	
31	purpose_house	0.042317	0.042317	
28	purpose_debt_consolidation	-0.041055	0.041055	
29	purpose_educational	-0.032206	0.032206	
27	purpose_credit_card	0.030400	0.030400	
32	purpose_major_purchase	-0.021624	0.021624	
36	purpose_renewable_energy	-0.019539	0.019539	
41	home_ownership_NONE	-0.009979	0.009979	
44	home_ownership_RENT	-0.007744	0.007744	
42	home_ownership_OTHER	0.001858	0.001858	

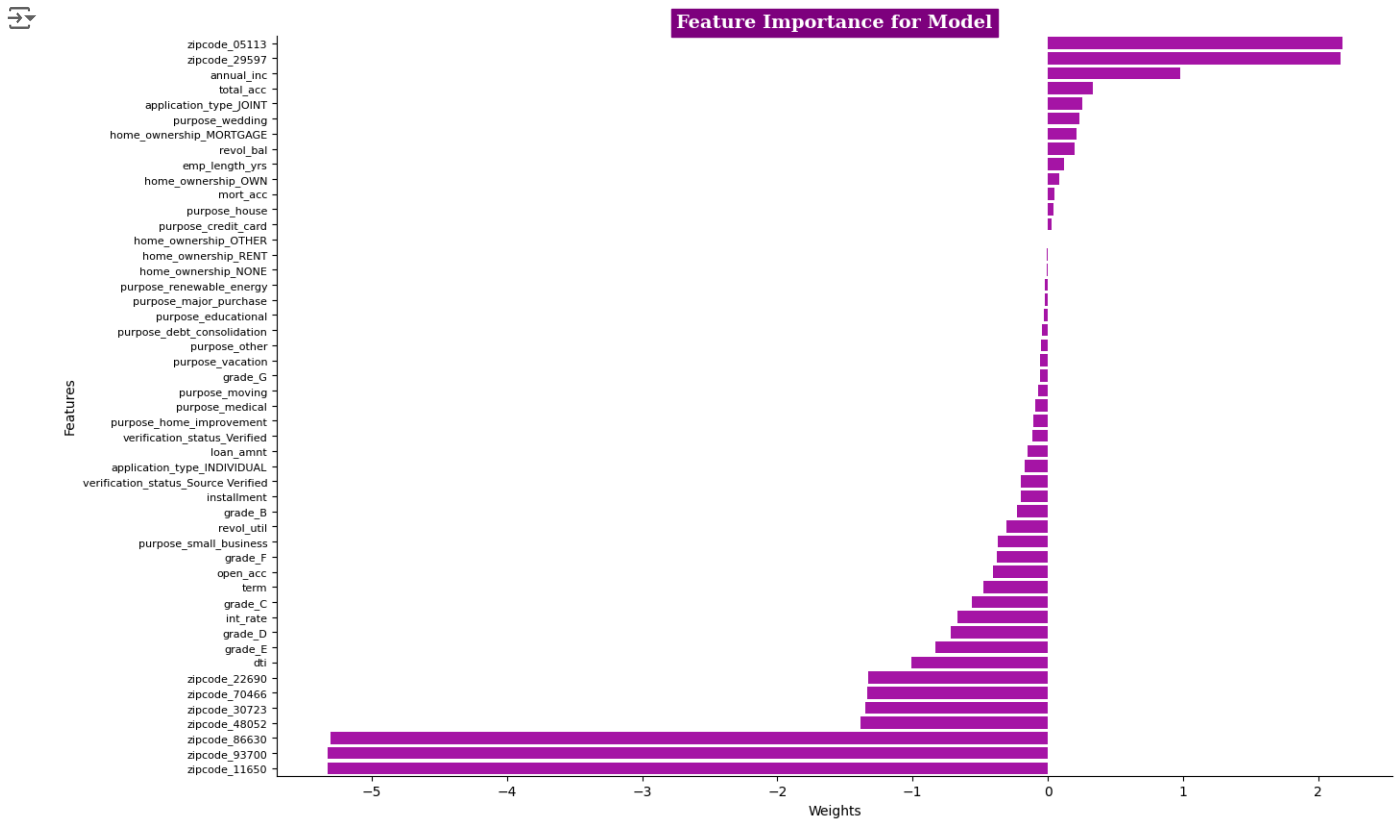
Next steps:

[Generate code with coeff_df](#)[View recommended plots](#)[New interactive sheet](#)

```

imp_feature = coeff_df.sort_values(by='Weights',ascending=False)
plt.figure(figsize=(15,10))
sns.barplot(y = imp_feature['Features'],
            x = imp_feature['Weights'],color='m')
plt.title("Feature Importance for Model",fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='purple',color='w')
plt.xlabel("Weights")
plt.yticks(fontsize=8)
plt.ylabel("Features")
sns.despine()
plt.show()

```



#Logistic Regression model intercept

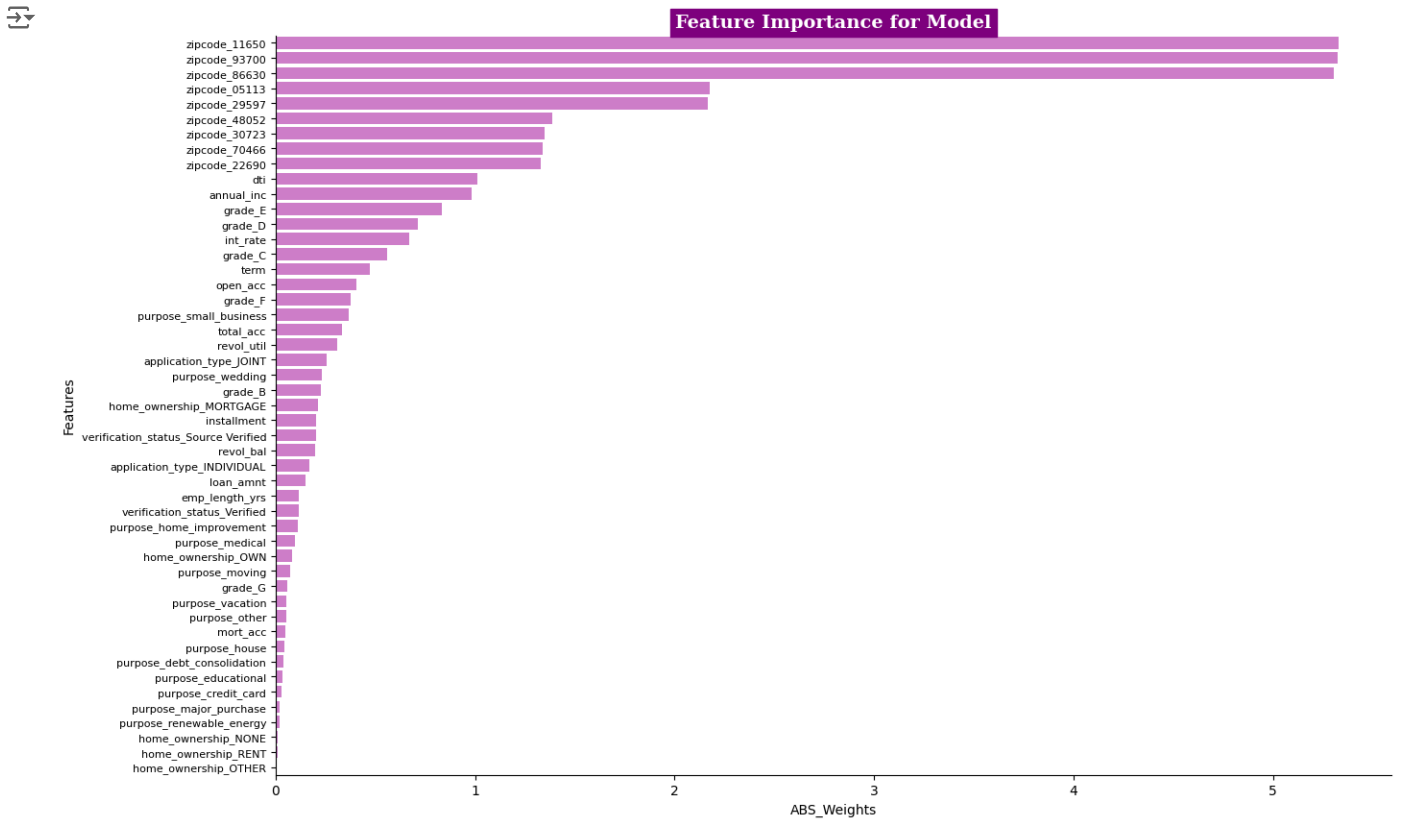
model.intercept_

array([4.19851815])

```

plt.figure(figsize=(15,10))
sns.barplot(y = coeff_df['Features'],x = coeff_df['ABS_Weights'],color='orchid')
plt.title("Feature Importance for Model",fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='purple',color='w')
plt.xlabel("ABS_Weights")
plt.yticks(fontsize=8)
plt.ylabel("Features")
sns.despine()
plt.show()

```



Observations:

- The model has assigned significant weight to the zip_code, Annual Income, grade features, indicating that certain zip codes strongly influence the prediction of defaulters.
- Features such as dti (debt-to-income ratio), open_acc (number of open accounts), and loan_amnt (loan amount) also have high positive coefficients, highlighting their importance in predicting default risk.
- On the other hand, several zip codes have large negative coefficients, suggesting that they are associated with a lower likelihood of default.

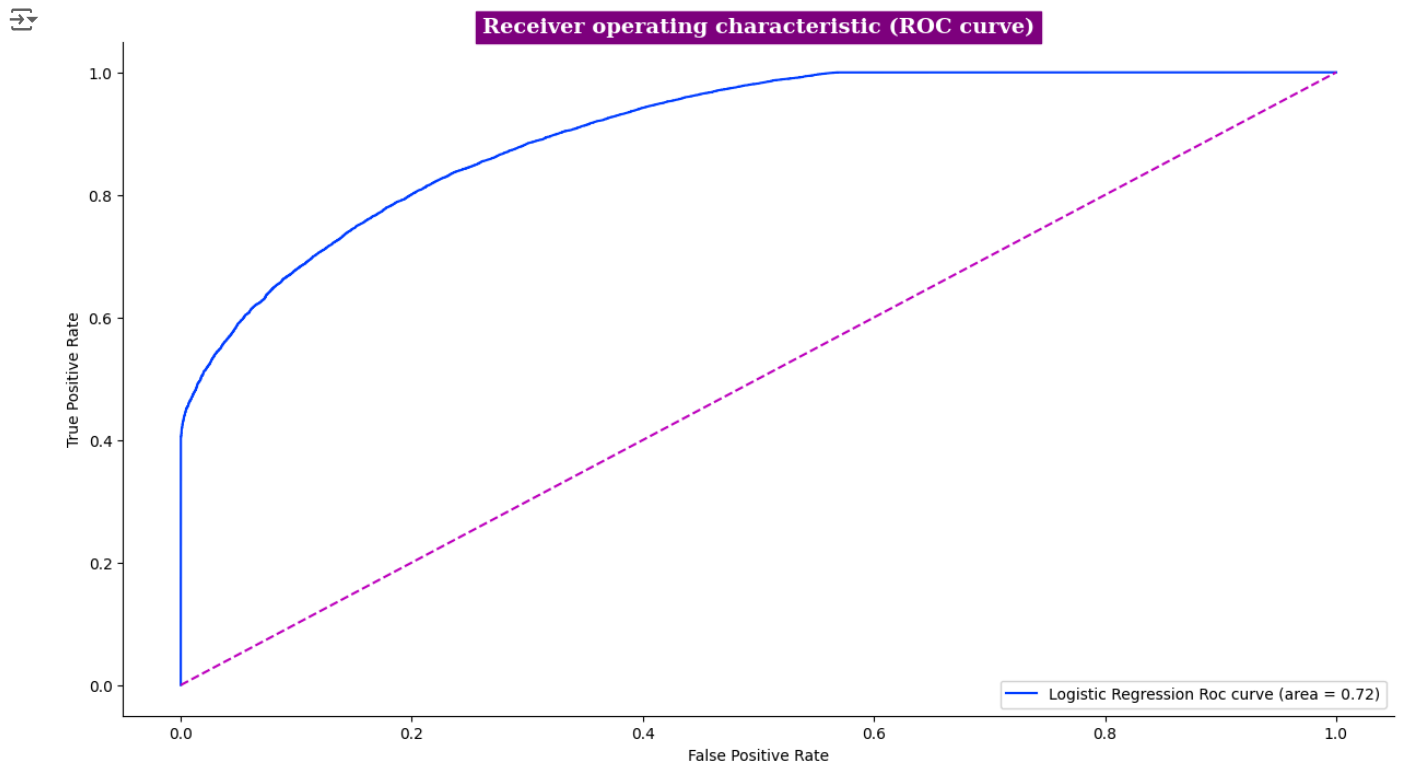
ROC AUC curve

```
# area under ROC curve
logit_roc_auc = roc_auc_score(y_test,y_reg_pred)

# Compute the false positive rate, true positive rate, and thresholds
fpr,tpr,thresholds = roc_curve(y_test,y_reg_pred_proba[:,1])

# Compute the area under the ROC curve
roc_auc = auc(fpr, tpr)

# plot ROC curve
plt.figure(figsize=(15,8))
plt.plot(fpr,tpr,label='Logistic Regression Roc curve (area = %0.2f)'% logit_roc_auc)
plt.plot([0,1],[0,1], 'm--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic (ROC curve)',fontfamily='serif',fontweight='bold',backgroundcolor='white')
plt.legend(loc="lower right")
sns.despine()
plt.show()
```



```
logit_roc_auc
```

```
0.7221566085466022
```

```
roc_auc = auc(fpr, tpr)
roc_auc
```

```
0.9036968310880946
```

🔍 Insights:

Trade-off in Performance: The ROC curve area, representing model performance, is 72%. This indicates that the model effectively distinguishes between classes 72% of the time.

- Ideally, we aim for a higher True Positive Rate (TPR) and a lower False Positive Rate (FPR) to ensure accurate predictions.
- The ROC curve illustrates that as True Positives increase, there's a simultaneous increase in False Positives.
- Misclassification: This trade-off implies that while identifying more Fully Paid customers, there's a heightened risk of misclassifying Charged Off customers as Fully Paid, potentially leading to Non-Performing Assets (NPAs).

These points emphasize the need to mitigate this risk:

- Reducing FPR while maintaining TPR is crucial to minimize misclassifications and associated risks.
- By shifting False Positives towards the left on the ROC curve, the model's overall performance, as measured by AUC, can improve.
- This improvement in AUC relies on maintaining a high True Positive Rate while reducing False Positives.

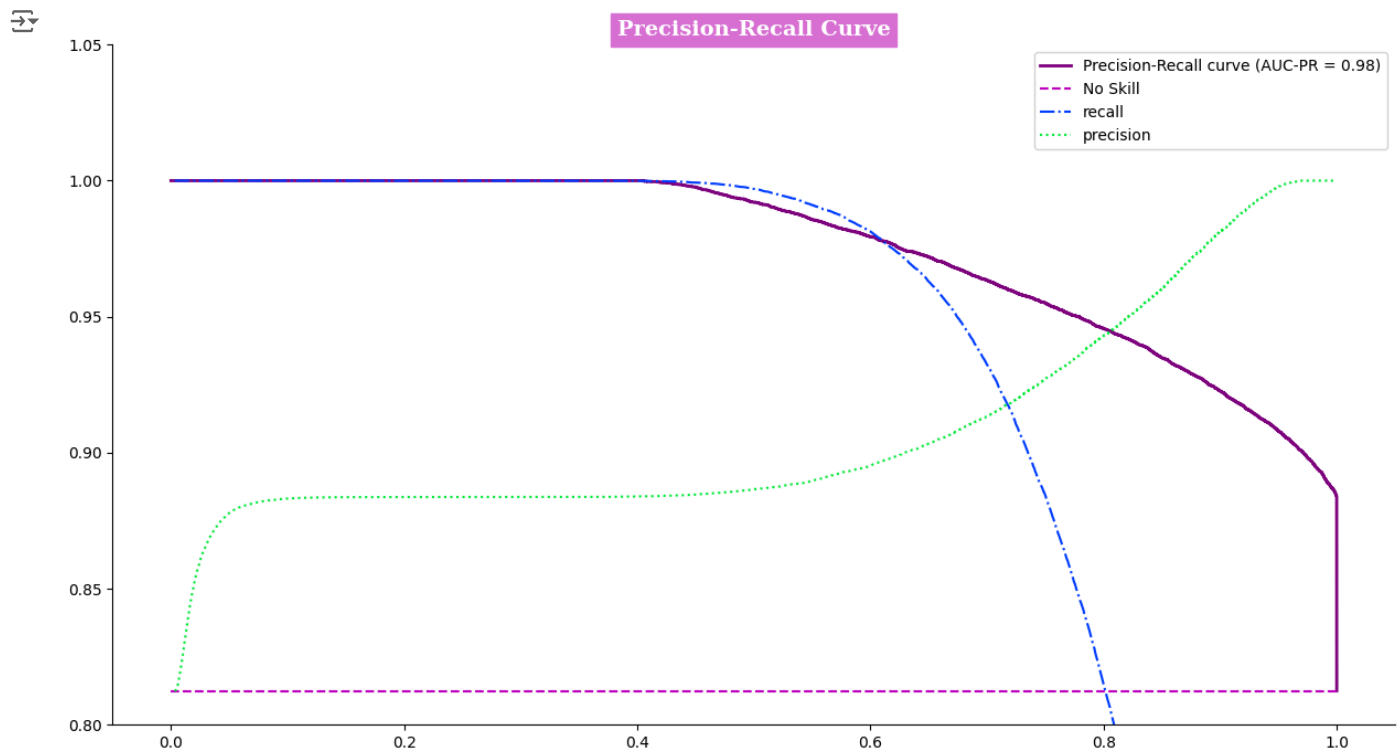
```
precision, recall, thresholds = precision_recall_curve(y_test, y_reg_pred_proba[:,1])
```

```
average_precision = average_precision_score(y_test, y_reg_pred_proba[:,1])
```

```
no_skill = len(y_test[y_test==1]) / len(y_test)
```

```
plt.figure(figsize=(15,8))
plt.plot(recall, precision, color='purple', lw=2, label=f'Precision-Recall curve (AUC-PR = {average_precision:.2f})')
plt.plot([0, 1], [no_skill, no_skill], linestyle='--', label='No Skill', color='m')
plt.plot(thresholds, recall[0:thresholds.shape[0]], label='recall',linestyle='-.')
plt.plot(thresholds, precision[0:thresholds.shape[0]], label='precision',linestyle='dotted')
# plt.xlim([0.0, 1.0])
plt.ylim([0.8, 1.05])
plt.title('Precision-Recall Curve',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='orchid',color='w')
```

```
plt.legend(loc='upper right')
sns.despine()
plt.show()
```



```
auc(recall, precision).round(3)
```

Observations:

Insight:

- The Area Under the Curve (AUC) for the precision-recall curve is 0.975. This high AUC value suggests that the model achieves excellent performance in distinguishing between positive and negative classes, showcasing strong precision-recall characteristics.
- **Precision-Recall Curve Superiority:** Precision-recall curves are pivotal, especially in imbalanced datasets, focusing on accurate predictions of the relevant class (Class 1 - Fully paid in this case).
- **Irrelevance of True Negatives:** Precision and recall computations disregard true negatives, simplifying focus to the correct prediction of Fully Paid customers.
- **AUC Strengthens Model Evaluation:** A high AUC (97.5%) underscores the model's robustness in distinguishing between classes, indicating its efficacy.
- **Precision Enhancement Priority:** Optimal model refinement centers on elevating precision by minimizing False Positives, vital for improving overall performance and mitigating risks.

```
# balanced Model
lr = LogisticRegression(max_iter=1000, class_weight='balanced')

lr_model = lr.fit(X_train, y_train)

print(classification_report(y_test, lr_model.predict(X_test)))

cm_bal = confusion_matrix(y_test, lr_model.predict(X_test))
cm_bal_df = pd.DataFrame(cm_bal, index=['Defaulter', 'Fully paid'], columns=['Defaulter', 'Fully paid'])
cm_bal_df
```


	precision	recall	f1-score	support
0	0.47	0.81	0.60	11678
1	0.95	0.79	0.86	50601
accuracy			0.79	62279
macro avg	0.71	0.80	0.73	62279
weighted avg	0.86	0.79	0.81	62279

	Defaulter	Fully paid
Defaulter	9468	2210
Fully paid	10586	40015

Next steps:

[Generate code with cm_bal_df](#)[View recommended plots](#)[New interactive sheet](#)

Observations from classification report:

Balanced model

- Precision : 95%
- Recall : 79%
- F1-score : 86%
- Accuracy : 79%

Insights:

- TN = 9466 (True Negative: Correctly predicted Charged Off)
- TP = 40028 (True Positive: Correctly predicted Fully Paid)
- FP = 2212 (False Positive: Predicted Fully Paid but actually Charged Off)
- FN = 10573 (False Negative: Predicted Charged Off but actually Fully Paid)
- Actual Negative (Charged Off) = 9466 + 2212 = 11678
- Actual Positive (Fully Paid) = 10573 + 40028 = 50601
- Predicted Negative (Charged Off) = 9466 + 10573 = 20039
- Predicted Positive (Fully Paid) = 2212 + 40028 = 42240

lr_model.intercept_

array([6.35580168])

Q6: Thinking from a bank's perspective, which metric should our primary focus be on..

- ROC AUC
- Precision
- Recall
- F1 Score

Ans:

From a bank's perspective, minimizing risks and maximizing profitability are paramount. ROC AUC (Receiver Operating Characteristic Area Under Curve) is indeed a crucial metric because it encompasses both True Positive Rate (TPR) and False Positive Rate (FPR)

- Bank's primary focus should be on ROC AUC , because bank needs to reduce FPR (False Positive Rate) and needs to increase the TPR (True Positive Rate).
- Maximizing TPR ensures that the bank correctly identifies customers who fully pay their loans (reducing False Negatives), while minimizing FPR ensures that the bank doesn't wrongly classify customers as fully paid when they're actually charged off (reducing False Positives).
- By optimizing ROC AUC, the bank can strike a balance between correctly identifying creditworthy customers and minimizing the risk of defaulters, thereby enhancing the overall performance and reliability of its credit scoring model.

Another approach:

- since I'm having High Recall value of 100% in Regularized model(most efficient model):

From a bank's perspective, the primary focus should be on minimizing risks while maximizing profitability. Therefore, the most relevant metric would be **Precision**.

- Precision represents the proportion of correctly predicted positive instances (e.g., customers who fully pay their loans) out of all instances predicted as positive. In the context of a bank, precision reflects the accuracy of identifying creditworthy customers who are likely to repay their loans. Maximizing precision ensures that the bank minimizes the number of false positives, which are instances where the bank incorrectly identifies customers as creditworthy when they are not. By prioritizing precision, the bank can reduce the risk of loan defaults and associated financial losses.
- While ROC AUC, Recall, and F1 Score are also important metrics, precision aligns closely with the bank's objective of minimizing risks and ensuring the quality of its loan portfolio.

🔥 Q7. How does the gap in precision and recall affect the bank?

Ans:

- To comprehend the errors made by a model, it's crucial to evaluate both false positives and false negatives, which are gauged through metrics like recall and precision. When recall is low, it poses a significant risk for the bank.
- So, the gap between precision and recall will affect the bank. As the gap widens, there will be increase in incorrect predictions.
- Good precision means less False Positives. i.e. Less NPA loan accounts.
- Good recall means less False Negatives. i.e. not loosing on good customer.

🔥 Q8. Which were the features that heavily affected the outcome?

Ans:

- Address(Zipcode), Annual_Income, Grade seems to be most important feature in our case.
- Loan duration term, Total Credit balance revol_bal, : Monthly debt vs. monthly income ratio dti, Interest int_rate also has high weights(coeffients) in the model .

🔥 Q9. Will the results be affected by geographical location? (Yes/No)

Ans:

- Yes, we can see that zip_code (Address) is a very important feature so geographical location has impact on our result.

✓ 💰🚀 Business Recommendations for LoanTap 🚀 💰📢

Optimize Loan Approval Strategy:

- Focus on maximizing the F1 score and area under the Precision-Recall Curve to effectively manage the precision-recall trade-off. This ensures identifying most defaulters while reducing false positives, enhancing risk management.

Model Improvement:

- Consider using more complex classifiers like Random Forests or XGBoost and perform hyperparameter tuning to enhance model performance and capture intricate relationships in the data.

Cross-Validation:

- Employed stratified k-fold cross-validation to ensure representative distribution of minority class in each fold, providing reliable estimates of model performance.

Policy Adjustments Based on Insights

- Scrutinize loans with lower grades more rigorously and consider adjusting interest rates to compensate for higher risk.
- Implement targeted strategies for high-risk zip codes, such as additional verification steps or higher interest rates.
- Evaluate small business loans with additional financial health checks and collateral requirements to mitigate default risk.

By implementing these recommendations, LoanTap can enhance their loan approval process, minimize the risk of NPAs, and ensure sustainable growth and financial stability.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.